



中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

学 校 中国石油大学（华东）

参赛队号 25104250018

队员姓名	1. 赵浩羽
	2. 蒋铭皓
	3. 田欣媛

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 多层协同优化的神经网络处理器核内调度算法研究

摘 要：

基于单指令多数据流架构的 NPU 因硬件设计简单、面效高等优势而备受推理任务的青睐。本文研究通用神经网络处理器下的核内调度问题。针对华为 Davinci 架构硬件平台在复杂计算图调度中面临的峰值内存控制、缓存资源管理和执行时间优化等多重挑战，建立多目标融合的自适应列表调度模型、硬件仿真框架模型、多层次协同内存管理模型和两阶段时间优化模型，实现从基础调度到性能优化的完整技术链条。

针对问题一，建立基于多目标融合的自适应列表调度算法，设计四层协同架构实现峰值内存驻留最小化。构建预测性缓冲区特征分析层，通过生存期效率分析、内存压力评估和关键路径权重计算实现精确特征提取；建立多目标动态优先级决策层，融合内存效率、执行效率和资源利用率实现智能调度决策；设计内存压力自适应调度层，通过压力感知和峰值邻域检测实现动态策略调整；构建分层协同全局搜索优化层，结合三阶段局部重排和二分搜索框架实现系统性优化。算法对于节点的时间复杂度为 $O(|N|^2)$ ，在六个示例上的优化效果为：Matmul_Case0 峰值内存降至 7,178 字节，Matmul_Case1 为 21,504 字节，FlashAttention_Case0 为 23,552 字节，FlashAttention_Case1 为 66,048 字节，Conv_Case0 为 29,216 字节，Conv_Case1 为 105,380 字节，相比传统贪心算法峰值内存平均降低 91.2%，最大优化幅度达 94%。本文提出的自适应列表调度算法在上述样例中的执行时间分别为 0.10s、9.69s、0.04s、0.47s、0.07s、16.66s，验证了算法的高效性和可扩展性。

针对问题二，构建 Davinci AiCore 硬件仿真框架并设计多层次协同的自适应内存管理算法，实现带约束的动态内存分配与缓存优化。首先建立层次化缓存抽象模型，定义连续性约束、非重叠约束和 L0 缓存独占约束的完整约束体系。在此基础上设计智能地址分配子系统，采用多策略融合分配器和碎片化感知策略，通过剩余生命周期、下次使用距离、碎片化贡献、未来分配压力和使用频率五个维度综合评估 SPILL 优先级。最后构建协同优化子系统，集成局部重调度机制和 COPY_IN 替代策略。在六个测试用例中应用上述优化，结果表明，算法总运行时间为 43.789 秒，成功管理 9,874 个缓冲区，完成 2,064 次 SPILL 操作，总额外数据搬运量为 306,206 字节。各用例具体数据搬运量分别为：FlashAttention_Case0:

9,020 字节、Conv_Case0: 68,314 字节、Matmul_Case0: 10,240 字节、FlashAttention_Case1: 50,544 字节、Matmul_Case1: 57,344 字节、Conv_Case1: 110,744 字节。缓存碎片化控制效果显著，L1 缓存平均碎片化率为 18.7%，UB 缓存平均碎片化率仅为 8.4%。考虑到模型的通用性，我们采用最优页面置换算法给出条件理论最优解分析，确保优化结果的合理性。

针对问题三，提出两阶段时间优化策略，实现执行时间与数据搬运量的多目标联合优化。第一阶段基于关键路径列表调度算法，通过关键路径余时计算和事件驱动调度策略优化节点执行顺序；第二阶段应用生命周期压缩、SPILL 预取和局部搜索技术，通过空洞填补和相邻节点交换实现精细调优。建立多目标权衡评估机制，为不同应用场景提供灵活的优化方案选择。在整体性能方面，相对于问题二基准，两阶段优化策略显著提升了时间性能。总执行时间从基准的 4,940,058 周期分别减少至 3,120,792 周期和 2,973,690 周期，实现 36.8% 和 39.8% 的优化幅度。与此同时，为获得时间性能提升，数据搬运量有所增加，总额外数据搬运量从 306,206 字节上升至 368,158 字节，且在二阶段优化上升至 1,085,957 字节，增幅分别为 20.2% 和 254.6%，可根据性能与精度权衡选择优化方案。在测试用例中，Matmul 类算子优化效果最为显著：Case0 执行时间降低 66.7% 至 99,488 周期，Case1 降低 62.6% 至 966,059 周期。FlashAttention Case0 通过第二阶段优化实现 32.6% 的性能提升，最终达到 49,988 周期。Conv Case0 也获得 6.3% 的改进，优化至 452,035 周期。在多目标权衡方面，算法的时钟周期平均降低 31.9%，且一阶段优化无明显搬运数据量激增，在时间优化与空间开销之间达到平衡，成功实现了多目标联合优化。

解决上述问题后，根据约束条件对结果进行严格校验，并给出优缺点分析与模型推广。

关键词： 神经网络处理器 硬件仿真 多目标优化 缓存管理 自适应调度

目录

1	问题综述	7
1.1	问题背景	7
1.2	数据分析	8
1.2.1	JSON 格式	8
1.2.2	CSV 格式	9
1.3	数据预处理	9
1.4	名词解释	10
1.5	拟解决的问题	11
2	问题分析	12
3	模型假设与符号说明	14
3.1	模型假设	14
3.2	符号说明	14
4	问题一的建模与求解	15
4.1	基础分析	15
4.2	专用模型 A-基于双层优先级策略的调度算法	16
4.2.1	约束条件	17
4.2.2	释放优先策略	18
4.2.3	最小分配策略	18
4.2.4	对算法的理论分析	19
4.3	专用模型 B-智能化多因子优化调度算法	19
4.3.1	缓冲区生命周期函数:	19
4.3.2	前瞻性内存预测函数:	19
4.3.3	多维度节点评分机制:	19
4.3.4	对算法的理论分析	20
4.4	专用模型 C-基于混合拓扑遗传优化调度算法	20
4.4.1	编码与拓扑保持层	21
4.4.2	目标评估层	21
4.4.3	遗传演化层	21
4.4.4	可行性修补与参数层	22
4.4.5	对算法的理论分析	22

4.5	通用模型-基于多目标融合的自适应列表调度算法	22
4.5.1	预测性缓冲区特征分析层	23
4.5.2	多目标动态优先级决策层	24
4.5.3	内存压力自适应调度层	25
4.5.4	分层协同的全局搜索优化层	26
4.5.5	算法复杂度分析	27
4.6	实验结果与算法性能分析	27
4.6.1	测试环境与数据集	27
4.6.2	实验统计结果分析	28
4.6.3	多算法对比实验	29
4.6.4	算法性能提升分析	30
4.6.5	内存使用模式分析	31
4.6.6	核心算法有效性分析	32
4.6.7	实验结果总结	32
5	问题二的建模与求解	33
5.1	基础分析与问题建模	33
5.2	Davinci AiCore 硬件仿真框架设计	33
5.2.1	缓存抽象模型	33
5.2.2	约束条件建模	34
5.2.3	动态状态转移机制	34
5.3	多层次协同的自适应内存管理算法	34
5.3.1	智能地址分配子系统	35
5.3.2	动态 SPILL 决策子系统	36
5.3.3	协同优化子系统	38
5.3.4	算法集成与复杂度分析	38
5.3.5	总额外数据搬运量计算	39
5.4	实验结果与性能分析	39
5.4.1	测试环境与数据集	39
5.4.2	主要指标结果	39
5.4.3	关键技术贡献分析	41
5.4.4	性能优势总结	43
5.4.5	特定条件下的理论最优解分析	44
6	问题三的建模与求解	46

6.1	基础分析与问题建模	46
6.2	两阶段优化策略总体框架	46
6.2.1	两阶段优化策略总体设计	46
6.3	优化阶段 1: 关键路径列表调度算法	47
6.3.1	关键路径余时计算	47
6.3.2	事件驱动调度策略	47
6.3.3	阶段 1 算法实现流程	47
6.4	优化阶段 2: 局部优化与空洞填补策略	48
6.4.1	生命周期压缩优化	48
6.4.2	SPILL 预取策略	48
6.4.3	局部搜索优化	48
6.5	执行时间计算与优化评估	48
6.6	多目标权衡与策略选择	49
6.7	算法复杂度分析	49
6.8	实验结果与性能分析	49
6.8.1	测试环境与基准对比	49
6.8.2	主要性能指标结果	49
6.8.3	执行时间优化效果分析	50
6.8.4	硬件资源利用率与流水线优化分析	51
6.8.5	优化策略贡献度分解分析	52
6.8.6	算法收敛性与稳定性分析	53
6.8.7	性能优势总结	53
7	模型评价	54
7.1	模型的优点	54
7.2	模型的缺点	54
7.3	模型的推广	54
	参考文献	55
	附录 A 图表目录	56
	附录 B 问题三方案二的结果	57
	附录 C Python 源程序	60
C.1	第一问程序	60

C.2 第二问程序	64
C.3 第三问程序	69

1 问题综述

1.1 问题背景

近年来，人工智能技术的不断突破，推动着深度神经网络在语音识别、图像处理、自然语言处理等领域的广泛应用。随着深度学习推理模型的规模和计算量持续增长，传统 GPU 和 CPU 出现了高功耗和高延迟问题，阻碍了神经网络在边缘设备的落地应用。因此，为满足高效推理的需求，业界开发了各类神经网络的专用加速芯片，通用神经网络处理器（Neural Processing Unit, NPU）便是其中的代表^[1]。一类 NPU 凭借其设计简单的单指令多数据流（Single Instruction Multiple Data, SIMD）架构，能够在功耗和空间受限的边缘端高效地执行推理任务，展现出卓越的并行计算能力。然而，SIMD 型 NPU 的核内适配与调度复杂，需要在细粒度计算图上决定执行顺序与缓存分配，并在多执行单元与多级缓存约束下优化调度策略。由于手工编排模板的泛化能力较弱，需要设计一种面向细粒度计算图的通用自动调度算法，促进 SIMD 架构 NPU 的规模化应用。

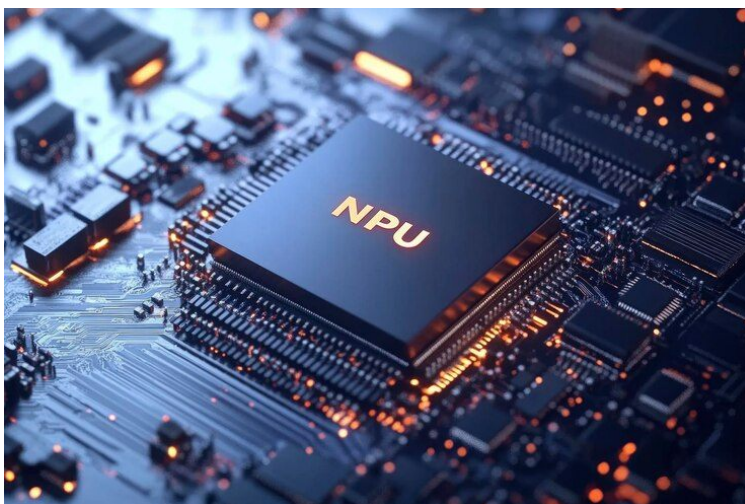


图 1 NPU 示例图

神经网络推理过程的计算主要依赖于算子这一基本单元，例如矩阵乘法 Matmul、卷积 Conv、注意力 Attention 等。而基本单元的执行效率会直接影响模型在硬件上的端到端性能。对于**华为 Davinci 架构**这类 SIMD 架构的 NPU，工程上通常将算子的完整计算流程，细化拆解为由底层硬件单元操作（如计算单元 Cube、Vector 和数据搬运单元 MTE）组成的细粒度计算图，之后采用手工或自动方式排布原子操作，得到执行序列。然而，这类计算图的高度异构性，导致仅靠人工经验编排计算单元流水变得十分困难、低效，难以适应日益复杂的模型计算场景。

上述核内调度问题面临诸多新挑战，需要一种通用调度算法，同时兼顾硬件异构并行性、有序执行约束，以及有限缓存容量和数据传输成本等多重限制条件。因此，开发一种通用的自动调度算法成为亟待解决的问题。

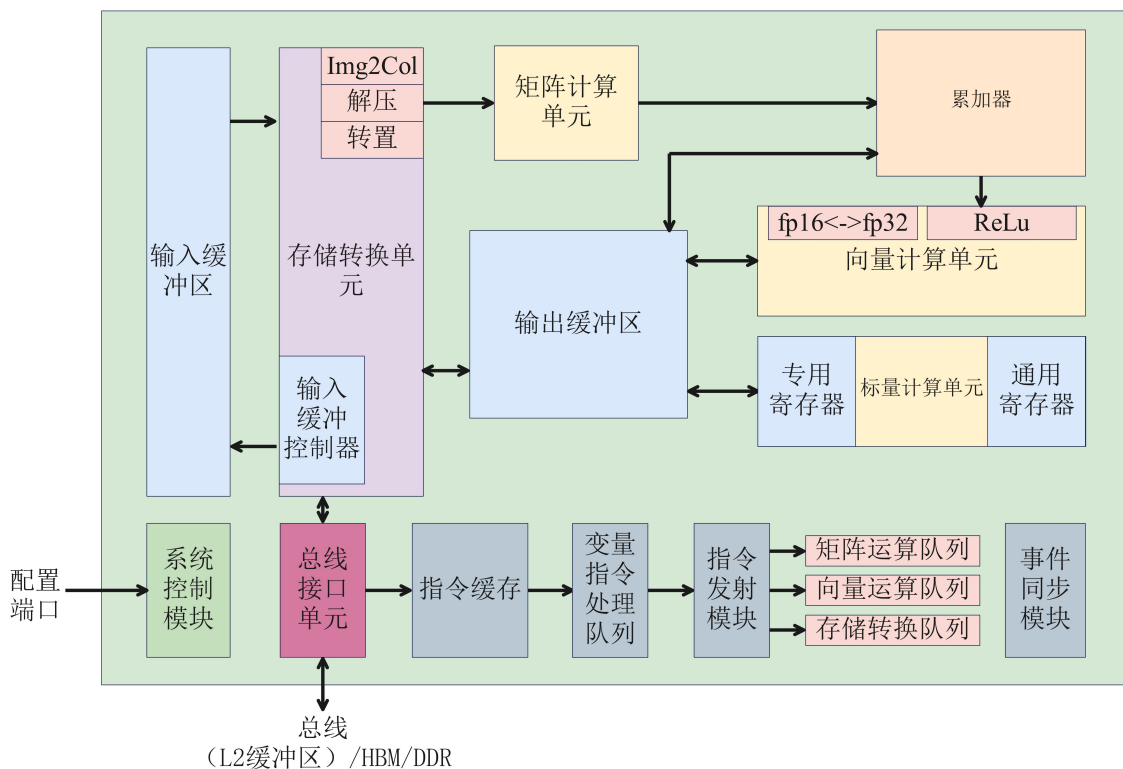


图 2 华为 Davinci 架构示例图

1.2 数据分析

本赛题提供的示例输入数据采用统一的图结构格式，分为 JSON 和 CSV 两种文件格式，描述六种任务的计算示例图模型（如 Matmul_Case0/1、FlashAttention_Case0/1、Conv_Case0/1）。这两种格式在数据内容上完全一致，差别仅在于结构组织和存储形式。根据实际建模过程中对数据解析和属性调用的需求，可以灵活选择使用 JSON 或 CSV 格式，为后续的图结构分析、节点调度优化等工作提供统一的数据基础。

1.2.1 JSON 格式

在 JSON 表示中，每个任务示例对应一个独立的文件，例如：任务名.json。该文件通过“顶级字段-子结构”的层级设计直接映射图模型的核心逻辑，包含仅有的两个顶级字段：**Nodes** 和 **Edges**。

(1) Nodes 列表：包含图中的所有节点，包括计算操作节点（如 CONV、MOVE、COPY_IN 等）和缓存管理节点（如 ALLOC、FREE 等）。每个节点以一个字典（JSON 对象）描述，并根据节点类型动态包含不同键值属性：

- 对于**操作节点**：包含 Pipe（执行管道，例如 MTE1、CUBE）、Cycles（执行周期）、Bufs（依赖的缓存列表）等属性，用于描述该操作的执行单元、耗费的周期和所依赖的缓存。

- 对于**缓存管理节点**：包含 BufId（缓存标识符）、Size（缓存大小）、Type（缓存级别，如 L1、L0B 等）等属性，用于标识和描述缓存空间的分配与释放情况。

(2) Edges 列表：定义节点间的有向依赖关系。每条依赖边以数组形式表示，例如 [9, 10] 表示源节点 9 的输出作为目标节点 10 的输入。通过此类二元数组即可清晰体现依赖方向，无需额外字段即可直接表征图中的有向边关系。

1.2.2 CSV 格式

以 CSV 表示时，节点信息和边信息被拆分存储在两个文件中：**一个节点信息文件**和**一个边信息文件**。

(1) 节点信息文件：以表格形式呈现节点属性。首行是字段标题，涵盖所有可能的节点属性列，例如：Id、Op、BufId、Size、Type、Pipe、Cycles、Bufs 等。不同类型的节点在对应行填入相关属性值，未涉及的字段则留空。例如：

- **ALLOC 节点**：需要填写 BufId、Size、Type 等字段（表示分配的缓存标识、大小和级别），而像 Pipe、Cycles 等执行相关字段对 ALLOC 来说无意义则留空。
- **COPY_IN 节点**：需要填写 Pipe、Cycles、Bufs 等字段（表示使用的执行单元、执行周期、依赖缓存），而与缓存分配无关的字段如 Size、Type 则空置。

通过这种方式，不同节点类型的数据在 CSV 中得到统一但灵活的表示，列字段与 JSON 的属性一一对应。

(2) 边信息文件：专注于图的依赖关系，仅包含两列：StartNodeId（起始节点 ID）和 EndNodeId（目标节点 ID）。每一行表示图中的一条有向边，直接对应 JSON 格式中 Edges 列表里的一个二元数组元素。这样的设计确保了 CSV 中的依赖关系表达与 JSON 保持完整一致，每条边都明确标识了源和目标，构成图的有向依赖结构。

1.3 数据预处理

本文采用所给数据的 JSON 格式，首先对六个示例计算图（Matmul、FlashAttention、Conv 各两例）的原始数据完成统一解析，以形成包含节点属性与依赖关系的标准化图数据结构，建立面向调度建模的辅助索引与标签。表1展示了附件中的数据信息。

在节点层面，依据 JSON 中 ‘Nodes’ 列表对每个节点进行逐条解析，建立完整的属性记录。核心字段包括：‘Id’（节点唯一标识）、‘Op’（操作类型，如 ALLOC、FREE、COPY_IN 等）、‘BufId’（缓存标识，用于关联分配与释放关系）、‘Size’（缓存大小，仅对缓存管理节点有意义）、‘Type’（缓存层级，如 L1、UB、L0A 等）、‘Pipe’（执行单元，如 CUBE、MTE1 等）、‘Cycles’（执行周期，用于衡量算子开销）、‘Bufs’（该操作依赖的缓存列表）。

在边关系层面，读取 JSON 的 ‘Edges’ 列表，获得每条依赖的起点与终点标识，并以此构建邻接表和逆邻接表这两类互补的索引结构，前者记录每个节点的后继集合，后者记录每个节点的前驱集合。两者共同刻画计算图的有向无环结构，使得任意节点的前置条件与

影响范围可以在 $O(\text{度})$ 的代价内被查询，同时也为拓扑可执行性判定、就绪集合维护以及生命周期分析提供了必要支撑。

表 1 附件数据信息统计表

任务名	节点数量	依赖边数量
Conv_Case0	2580	3869
Conv_Case1	36086	85653
FlashAttention_Case0	1716	2712
FlashAttention_Case1	6952	11184
Matmul_Case0	4160	7104
Matmul_Case1	30976	55040

基于节点的功能属性，进一步将全体节点划分为三类：缓存申请（ALLOC）、缓存释放（FREE）以及一般操作节点（计算与搬运等）。其中 ALLOC/FREE 直接改变驻留曲线，而一般操作节点通过改变 FREE 的就绪性间接影响峰值。考虑到目标平台的存储层次结构，本文对缓存相关节点的 ‘Type’ 字段进行重点标注，当类型为 L0A、L0B、L0C 等片上高速层级时，记为高优先级缓存节点。

数据预处理过程先导入六个示例任务的 JSON 格式，逐条解析 ‘Nodes’，并为每个节点建立包含 ‘Id’、‘Op’、‘BufId’、‘Size’、‘Type’、‘Pipe’、‘Cycles’、‘Bufs’ 在内的属性记录；之后读取 ‘Edges’ 并生成依赖关系列表，据此同时构建邻接表与逆邻接表，用于后续的就绪集合维护与生命周期分析；然后按照操作性质将节点划分为 ALLOC、FREE 与一般操作三类；在此基础上，对缓存相关节点检查其 ‘Type’ 值，若属于 L0A/L0B/L0C 等高速层级，则附加“高优先级缓存”标记；最后，将标准化的节点表、边表、索引结构与分类标签统一存储，形成后续图结构分析与调度优化的输入基础。

1.4 名词解释

- **算子（如矩阵乘 Matmul、卷积 Conv、注意力 Attention 等）**：算子作为一种计算操作手段，本质上是函数空间的相互映射，并且可以推广到任何空间，在深度学习领域的相关计算中，算子表示对输入数据执行特定数学运算，产生输出的过程。

- **SIMD 架构硬件平台（华为 Davinci 架构平台）**：单指令流多数据流架构采用一个控制器，同时管理多个处理器，操作着一组数据，能够实现空间并行性的一种框架，适合图像处理、科学计算以及深度学习等高度规则的数据并行任务。本文特指的华为 Davinci 架

构平台，是华为公司为昇腾 AI 处理器所设计的特定域架构，能够专注于神经网络推理训练等 AI 工作负载任务，其核心架构是异构计算单元和分层存储设计。

- **硬件单元操作：**硬件单元操作是硬件功能中的基础构建块，拥有确切的输入、输出和时序特征，在数字电路中借助特定功能单元在固定周期内执行基本计算和逻辑动作。

- **细粒度计算图：**细粒度计算图能够拆解计算任务，得到如单个乘法、加法、数据移动等原子操作单元，基于依赖关系构建有向无环图，图中的节点表示基本操作，边表示数据流向和依赖。

- **高度异构性：**计算机内部系统中集成着的多种架构特质、功能特定且指令集可能互不兼容的计算单元，针对具体计算任务进行硬件级优化。

- **调度算法：**系统资源管家操作系统，在面临多个进程请求使用有限资源的情况时，需要采取对应手段选择合适的进程占用资源，这个过程就称为调度。为达到不同的系统目标，研究人员制定了不同的调度算法。

- **多级缓存：**为平衡中央处理器（CPU）的高处理速度和主内存（DRAM）的缓慢速度，在两者之间设置包含多个拥有不同级别、容量和速度的静态随机存取存储器（SRAM）的层次结构。现代 CPU 的常用三级缓存结构，少量极快但昂贵的 L1 缓存（Level 1 Cache）用于存储 CPU 立即需要的数据，大量较快且便宜的 L2、L3 缓存（Level 2 Cache、Level 3 Cache）存储近期可能用到的数据，能够较好地平衡速度、容量和成本。

- **数据搬运（开销）：**计算机中的 CPU、GPU 以及 NPU 中的核心等计算单元，在执行计算任务时，需要在计算机存储层次结构的不同层级之间移动输入、权重以及中间结果等数据，转移数据的过程称为数据搬运。而在数据搬运过程中，消耗的时间和能量，便是数据搬运开销。

- **有向无环图（DAG）：**图结构是由若干个代表不同元素或实体的节点，以及连接节点的边组成。若每条边上标有箭头从一个节点指向另一个节点，表示节点间的依赖关系、先后顺序或信息流方向，并且从任意节点出发沿着有向边依次遍历，回不到出发节点，这种“有方向、无回路”的图称为有向无环图，任务调度、拓扑排序、数据依赖分析等场景中有着广泛应用。

- **两类节点：**

特定硬件系统往往包含两类节点，一类是操作节点，部署于硬件计算单元（如 CPU 的算术逻辑单元、GPU 的流处理器、专用运算模块等）内部，执行具体的计算任务，包括数学运算和逻辑任务；另一类是缓存管理节点，独立运行在存储控制单元中，区别于操作节点，其负责缓存资源的动态分配、释放和调度。

1.5 拟解决的问题

为综合权衡计算图的依赖关系、指令有序执行、多单元并行性以及缓存容量和地址空间约束等各方面要求，需建立数学模型设计一种自动调度算法解决以下问题：

问题一：最小缓存驻留调度

(1) 给定一个计算图 DAG，包含操作节点和缓存管理节点两类节点。图中每个节点附带执行单元、执行周期、所依赖的缓存等属性，边表示节点间的先序约束。

(2) 由于硬件多级缓存容量有限，需要在保证依赖关系的条件下，确定一个覆盖所有节点的可行调度序列，尽可能减小调度过程中驻留缓存峰值。

(3) 针对所有给定的示例计算图，输出每个图的节点调度顺序 (schedule.txt)，同时计算出该调度下的最大缓存驻留量，为后续缓存分配提供基础。

问题二：缓存分配与换入换出

(1) 基于问题一得到的调度序列，考虑硬件实际缓存容量，比如 L1、UB、L0A、L0B、L0C 等均为有限区间，结合硬件缓存资源大小给出约束。

(2) 实际分配缓存地址时，可能会导致并发缓冲区需求超过硬件容量的情况，需要引入换入换出操作 (SPILL)，将部分缓冲区临时写回 DDR，后续需要时再加载。本问题的核心是在缓存容量限制的情况下，如何完成所有缓冲区的分配，且尽量减少额外数据搬运量。

(3) 针对每个计算图，给出缓存地址分配方案 (memory.txt)，并记录换入换出操作 (spill.txt)。基于不同分配策略计算出调度过程的总额外数据搬运量。

问题三：性能优化策略

(1) 基于已有的调度顺序和缓存分配方案，考虑执行单元之间可以并发工作，需要在不显著增加数据搬运开销的前提下，尽量缩短计算图的总运行时间。

(2) 问题二所得到的方案侧重于减少数据搬运量，但整体运行时间可能偏长。本问题要求在额外数据搬运量不显著增加的前提下，通过改进调度顺序或优化缓存分配方式，降低总执行时间，从而提高调度方案的综合性能。

(3) 针对各示例计算图，输出新的调度顺序、缓存分配与 SPILL 操作结果，并计算两类核心指标：总运行时间和总额外数据搬运量。通过与问题二的结果对比，展示性能提升效果，并以文本附件形式提交 (schedule.txt、memory.txt、spill.txt)。

2 问题分析

通用神经网络处理器下的核内调度问题，可以概况为在空间、时间受限下，从有向无环图 (DAG) 中得到满足拓扑序的调度序列。该类问题属于资源受限项目调度问题 (RCPSP) 的衍生类型，是典型的 NP-hard 问题。本文基于华为 Davinci 架构平台，需要在硬件架构约束下求得初始的调度序列；再进一步考虑因缓存资源死锁而产生的 spill 分配策略；最终，将总运行时间纳入考量，以获得完整的调度算法。本研究总体路线如图3所示。

问题一分析：对于问题一，已知为输入为计算图以及每个节点需要的缓存大小，需要满足拓扑约束和内存一致性约束，最终输出一个节点调度序列 S，且使得 S 在执行过程中使用的最大驻留缓存容量最小，并且给出对于节点总数的时间复杂度分析。考虑到调度算

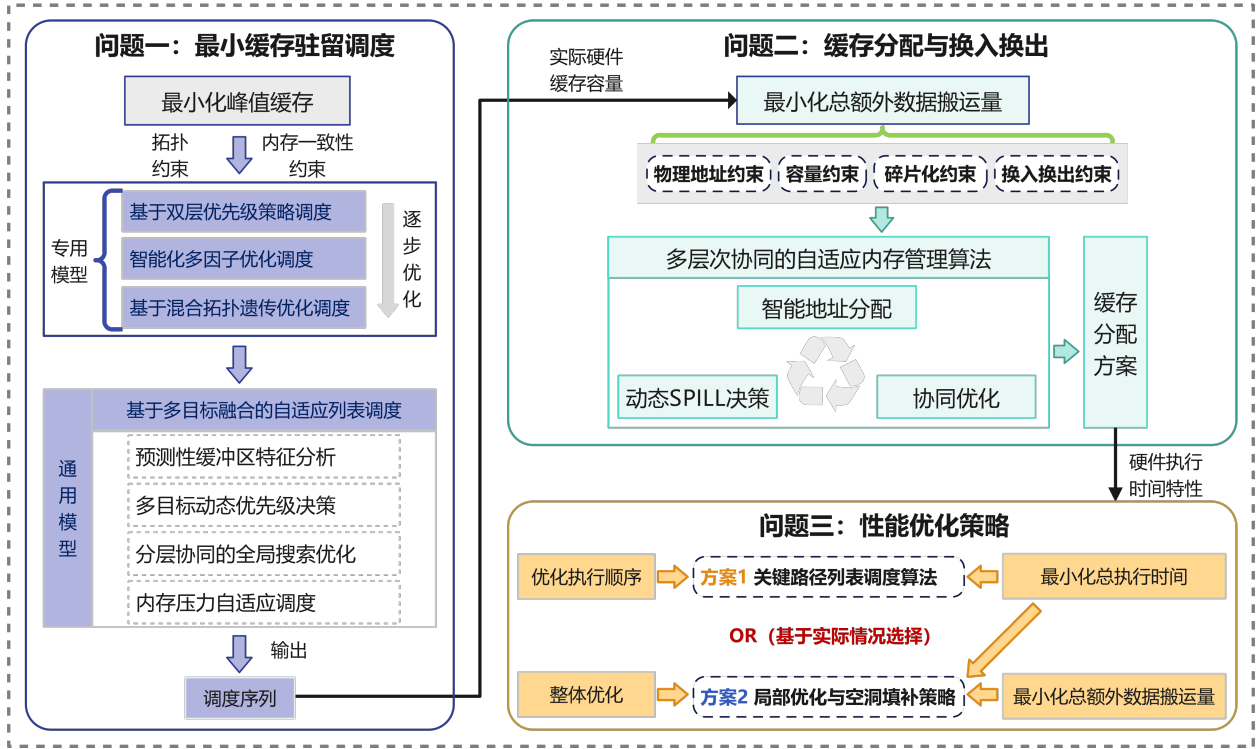


图3 全文技术路线图

法通用性，问题一的模型需要面临卷积、矩阵乘法和注意力机制等包含大量节点与边的典型算子大规模计算图。如果仅采取贪心算法，可以快速求得可行解，但距离最优解仍有较大差距；如果使用元启发优化方法，可以在多轮迭代下也有可能逼近或者得到最优解，但是面临结果随机、泛化性较差等、计算开销大等问题。因此，提出一种基于架构机理建模的、具有泛化性、优化效率高的解决方案是非常有必要的。

问题二分析：对于问题二，在得到符合约束的节点调度序列后，将硬件缓存资源的不同物理空间大小纳入考虑，最终输出每个缓冲区对应的地址偏移、分配过程需要的 SPILL 操作列表以及总额外数据搬运量结果。问题二本质上是一个带约束的动态内存管理问题，其核心挑战在于如何刻画不同约束条件之间的耦合关系，并设计智能地址分配子系统，最后给出缓存分配方案。

问题三分析：问题三需要联合使用前两问的调度算法，综合考虑执行时间、总额外数据搬运量等优化目标，建立完整的调度算法。问题三涉及多目标优化问题，主要难点为复杂多目标的权衡问题：更少的执行时间往往需要更激进的调度策略，可能增加缓存压力；而保持较低的数据搬运量则倾向于保守的缓存管理，两者存在本质冲突。根据实际情况，选择优化目标是降低算子的总运行时间，还是寻求两项指标的联合优化。因此需要提出两个方案，可以基于工程实际情况进一步选择。

需要重点强调的是，在所有问题的求解中，不只仅限于**特定算子或者示例数据**，而考虑模型面对未知数据的**泛化性与通用性**是必要的。

3 模型假设与符号说明

3.1 模型假设

1. 数据传输假设：假设数据传输是无损的，没有错位的；且忽略因信号传播延迟、时钟抖动等原因造成的物理延迟。本假设有助于本团队构建一个可控的仿真环境，将视角更加专注于调度和内存管理策略本身。

2. 问题一的特定假设：在问题一中，不考虑实际硬件上 L1 和 UB 缓存的长度限制，且认为 ALLOC 和 FREE 节点本身不消耗执行周期。这是因为问题一仅关注调度顺序，且不涉及缓存分配策略和总运行时间。

3.DDR 假设：假设 DDR 容量无限且访问延迟固定，即当缓存空间不足时，数据可以无限制地 SPILL 到 DDR（核外内存），且 SPILL 操作的延迟和带宽是固定且可预测的。本假设旨在简化了 SPILL 操作的建模。

4. 缓存分配的连续性要求：假设每个缓冲区在分配过程中必须获得连续的物理地址空间。如果本假设不成立，就不会出现内存碎片化问题，分配策略会有所不同。

5. 执行单元的独立性与无竞争：除了明确的资源约束（如 L0 缓存独占性、Pipe 的独占使用），不同执行单元之间的操作是相对独立的，不考虑它们之间可能存在的微架构竞争（如共享总线带宽、寄存器文件冲突等）。本假设旨在简化了并行执行的建模，使得可以独立计算不同 Pipe 上的任务时间。

3.2 符号说明

本文使用的主要符号如表2所示。

表 2 符号说明

符号	说明	符号	说明
$G = (V, E)$	计算图, V 为节点集合, E 为边集合	$Op(v)$	节点 v 的操作类型
$v \in V$	计算图中的单个节点	$BufId(v)$	节点 v 关联的缓冲区标识符
$(u, v) \in E$	计算图中的有向依赖边	$Size(v)$	节点 v 分配/释放的缓冲区大小
$Type(v)$	节点 v 关联的缓存类型	π	调度序列, $\pi(t)$ 表示第 t 步执行的节点
$InDegree(v)$	节点 v 的入度	$pos(v, \pi)$	节点 v 在调度序列 π 中的位置索引
$\mathcal{M}(t)$	调度前缀 t 时刻的缓存驻留量	$\Delta M(v)$	节点 v 的内存净变化量
$\mathcal{M}_{peak}$	调度过程的最大缓存驻留量	B	计算图中的缓冲区集合
$\mathcal{L}(b)$	缓冲区 b 的生命周期	$F(v, d)$	节点 v 深度为 d 的前瞻性内存预测函数
$S(v)$	节点 v 的综合评分	C	缓存类型集合
$Cap(c)$	缓存类型 c 的容量	$\mathcal{R}_c$	缓存 c 中的已分配区域集合
$\mathcal{F}_c$	缓存 c 中的空闲区域集合	D_{total}	SPILL 操作的总额外数据搬运量
T_{total}	计算图的总执行时间	$L_{bottom}(v)$	节点 v 到图终点的最长路径长度
$Load_{pipe}(p, t)$	执行单元 p 在时刻 t 的负载量	$\mathcal{J}$	多目标联合优化函数

4 问题一的建模与求解

问题一主要分析的是通用神经网络处理器的核内调度问题, 核心的目标是要构建出合理的核内调度算法, 为后续调度的空间优化工作与世间优化工作提高有效的支撑。

4.1 基础分析

通用神经网络处理器的核内调度问题可以概括为在资源受限下的有向无环图 (DAG) 调度问题, 该类问题属于**资源受限项目调度问题 (RCPSP)**的衍生类型。在该类型中, 模型的计算流程被转换为 DAG 的形式, 其中节点不仅代表了在不同硬件单元上运行的计算算子, 还包含了申请和释放中间数据缓冲区的内存管理操作。调度任务目的是在满足拓扑依赖关系与硬件资源限制的前提下, 明确各节点的运行次序与资源分配策略, 从而实现各计算单元间的并行运行, 并最大限度的降低因缓存容量不足所导致的数据搬运与交换损耗。

问题一的求解关键在于解决**峰值内存驻留最小化**的排序问题。从目标本质分析, 该问题需要在所有可行的拓扑序列中筛选出一种调度策略, 使得并发生存所需要的总缓冲区容量峰值降至最低, 该类问题属于典型的 NP-hard 问题。针对卷积、矩阵乘法和注意力机制

等典型算子包含大量节点与边的大规模计算图，直接采用精确的方法获取全局的最优解往往比较困难。因此，在实际的应用场景中，通常需要借助启发式方法或分解策略来得到可行的解决方案。

为方便后续的表述，记任一可行拓扑序列为 $\pi(1), \dots, \pi(|V|)$ 。其中， V 表示节点集合， $\pi(t)$ 表示第 t 个执行的节点。定义前缀 t 时刻的驻留量 $\mathcal{M}(t)$ 为：

$$\mathcal{M}(t) = \sum_{i \leq t} \Delta(\pi(i)) \quad (4.1)$$

其中 $\Delta M(v)$ 是节点 v 对内存的净变化量：

$$\Delta M(v) = \begin{cases} +\text{Size}(v) & \text{if Op}(v) = \text{ALLOC} \\ -\text{Size}(v) & \text{if Op}(v) = \text{FREE} \\ 0 & \text{otherwise} \end{cases} \quad (4.2)$$

峰值内存使用量为：

$$\mathcal{M}_{\text{peak}} = \max_{t=1,2,\dots,n} M(t) \quad (4.3)$$

优化目标是在满足拓扑约束和内存一致性约束的前提下，找到一个调度序列 π 使得 $\min_{\pi} \mathcal{M}_{\text{peak}}$ 。

根据题目的相关说明，本题附件所提供的六个示例计算图仅用于思路验证。在实际的应用场景中，芯片开发人员无法预先掌握用户可能的全部原子操作，因此我们需要搭建一套基于确定性机制的调度方案，该方案兼顾调度效率与通用性，且不依赖于启发式方法或机器学习技术。

基于上述的考虑，问题一在求解过程中提出了以下两类模型方案：

- **专用性模型：**该模型以附件中的六个示例计算图为主要测试对象，结合各原子操作已知的特定信息进行针对性的建模工作。其目标是在给定的条件下尽可能取得全局最优解，主要用于方法有效性的验证。
- **通用性模型：**该模型不依赖于示例中任何特定的计算图结构算子类型，而是通过对机制的严格分析与推导，设计了具备良好泛化能力的调度策略，适用于结构未知的计算图任务调度场景，更具说服力。

最终，我们在综合评估两类模型的策略后，将通用性模型作为本问的核心方案。

4.2 专用模型 A-基于双层优先级策略的调度算法

在图相关的任务推理与任务调度中，计算图通常表示为 DAG 形式，其中节点代表基础的内存分配、计算和释放等操作，边代表操作间的依赖关系。因此，为了高效管理和使用内存资源，减少峰值内存使用量，本节提出了一种**基于双层优先级策略的贪心调度算法**，该算法通过优先调度内存释放操作和谨慎选择内存分配操作，来优化内存的使用。

设计算图为 $G = (V, E)$ ，其中：

- $V = \{v_1, v_2, \dots, v_n\}$ 表示节点集合。
- $E = \{(u, v) \mid u, v \in V\}$ 表示依赖关系边集合。

每个节点 $v_i \in V$ 具有以下属性：

- $Op(v_i) \in \{ALLOC, FREE, COMPUTE\}$ ：操作类型。其中 ALLOC 表示分配内存缓冲区，FREE 表示释放内存缓冲区，COMPUTE 表示计算操作。这些操作不直接影响内存但可能依赖缓冲区数据；
- $Size(v_i) \geq 0$ ：内存需求量。仅对于 ALLOC 和 FREE 节点有意义，表示申请或释放的数据缓冲区长度。
- $BufId(v_i)$ ：缓冲区标识符。用于关联 ALLOC 和 FREE 节点，以确保每个缓冲区的分配和释放操作能够正确配对，并避免出现内存泄漏。

本算法主要采用**双层贪心策略**：以最小化分配内存为原则，第一层策略目标是优先调度 FREE 节点以释放内存，第二层策略是在第一层测率之后已经无 FREE 节点时调度 ALLOC 节点，目的是平滑内存使用曲线，降低峰值内存。

算法 1 双层优先级策略调度算法

输入：节点集合 N ，边集合 E

输出：调度序列 π^* ，内存峰值 M^*

- 1: 初始化： $\pi^* \leftarrow \emptyset$ ，当前内存 $current_mem \leftarrow 0$ ，峰值 $M^* \leftarrow 0$
 - 2: Build dependency graph, compute in-degrees of all nodes; available $\leftarrow$ 入度为 0 的节点
 - 3: **while** available $\neq \emptyset$ **do**
 - 4: **if** $\exists$ FREE 节点 $\in$ available **then**
 - 5: 选择 available 中 Size 最大的 FREE 节点作为 sel // 优先释放大容量缓冲区，减少内存驻留
 - 6: **else if** $\exists$ ALLOC 节点 $\in$ available **then**
 - 7: 选择 available 中 Size 最小的 ALLOC 节点作为 sel // 次选小容量分配，降低内存峰值
 - 8: **else**
 - 9: 选择任意操作节点 (compute/move) 作为 sel
 - 10: Update $current_mem$ (ALLOC 节点加 Size, FREE 节点减 Size)
 - 11: $M^* \leftarrow \max(M^*, current_mem)$ // 更新当前最大内存驻留
 - 12: 将 sel 加入 π^* ，从 available 中移除 sel
 - 13: 更新 sel 邻接节点的入度；入度变为 0 时，加入 available // 解锁依赖节点
 - 14: **return** (π^*, M^*)
-

4.2.1 约束条件

根据题目的要求，本节我们提出的算法模型必须满足以下约束条件以确保正确性：

- **拓扑约束**：由于计算图属于 DAG 形式，因此必须遵守节点间的依赖关系。在调度序列中，对于任意边 $(u, v) \in E$ ，节点 u 必须在节点 v 之前执行，以确保数据流依赖：

$$pos(u) < pos(v), \quad \forall (u, v) \in E \quad (4.4)$$

其中 $\text{pos}(v_i)$ 表示节点 v_i 在调度序列中的位置索引。

• **内存一致性约束**: 每个缓冲区的 ALLOC 操作必须在对应的 FREE 操作之前执行, 避免出现内存泄漏或非法访问等情况。对于每个缓冲区 b , 设 $\text{ALLOC}(b)$ 和 $\text{FREE}(b)$ 分别表示分配和释放缓冲区 b 的节点, 则有:

$$\text{pos}(\text{ALLOC}(b)) < \text{pos}(\text{FREE}(b)), \quad \forall \text{buffer } b \quad (4.5)$$

构建计算图时, 配对关系通常通过缓冲区标识符进行维护, 而上述约束需要在调度过程中维护节点的入度来完成。

综上所述, 拓扑约束是只有当前节点的所有前驱节点都被调度后, 该节点才会转为可调度的状态。内存一致性约束是通过缓冲区的依赖边来隐含确保的, 比如 FREE 节点通常依赖于使用该缓冲区的 COMPUTE 节点, 而 COMPUTE 节点依赖于 ALLOC 节点。

4.2.2 释放优先策略

当存在可调度的 FREE 节点时, 算法会从中优先选择释放内存最大的节点, 减少当前的内存使用量, 为后续操作提供更多可用的内存空间, 一定程度上降低了整体内存的占用情况。

设 $\mathcal{F}_{\text{ready}}$ 为当前可调度的 FREE 节点集合, 即入度为 0 的 FREE 节点:

$$\mathcal{F}_{\text{ready}} = \{v \mid \text{Op}(v) = \text{FREE} \wedge \text{InDegree}(v) = 0\} \quad (4.6)$$

其中, $\text{InDegree}(v)$ 表示节点 v 的当前入度, 即未调度的前驱节点数量。

算法选择节点 v^* 为:

$$v^* = \arg \max_{v \in \mathcal{F}_{\text{ready}}} \text{Size}(v) \quad (4.7)$$

这一策略的目的在于尽早释放较大的缓冲区, 从而减少内存的过度累积, 以此达到优化内存的目的。

4.2.3 最小分配策略

当没有可以直接调度的 FREE 节点时, 算法转而选择 ALLOC 节点。此时, 会优先选择分配内存最小的节点, 让内存使用量缓慢增加, 避免内存峰值过早地出现。

形式化地, 设 $\mathcal{A}_{\text{ready}}$ 为当前可调度的 ALLOC 节点集合:

$$\mathcal{A}_{\text{ready}} = \{v \mid \text{Op}(v) = \text{ALLOC} \wedge \text{InDegree}(v) = 0\} \quad (4.8)$$

算法选择节点 v^* 作为:

$$v^* = \arg \min_{v \in \mathcal{A}_{\text{ready}}} \text{Size}(v) \quad (4.9)$$

该策略类似于最短作业优先算法, 优先处理内存需求较小的分配操作。

4.2.4 对算法的理论分析

基于上述分析，我们在算法1中展示双层优先级策略算法的伪代码。该算法首先对任务 DAG 进行拓扑排序（时间复杂度 $O(N + E)$ ），然后为每个任务计算双层优先级，并对任务列表按优先级排序 $O(N \log N)$ 。最后，按排序顺序依次为每个任务分配执行资源 $O(N)$ 。综合各步骤，算法相对于节点总数 N 的时间复杂度为 $O(N^2)$ 。

4.3 专用模型 B-智能化多因子优化调度算法

尽管 4.2 节采用了双层贪心策略，但我们发现其与最优解之间的差距仍较为显著。为此，我们引入了多维度评分机制与前瞻性预测策略，进一步优化 4.2 节的算法，提出了智能化多因子优化调度算法。在原有 DAG 模型基础上，引入以下扩展：

4.3.1 缓冲区生命周期函数：

$$\mathcal{L} : B \rightarrow N \quad (4.10)$$

其中 B 为缓冲区集合， $\mathcal{L}(b)$ 表示缓冲区 b 从 ALLOC 到 FREE 的依赖路径长度。

对于缓冲区 $b \in B$ ，其生命周期定义为从分配节点到释放节点的最短依赖路径长度：

$$\mathcal{L}(b) = \min_{\pi} |\pi(\text{ALLOC}(b), \text{FREE}(b))| \quad (4.11)$$

其中 $\pi(\text{ALLOC}(b), \text{FREE}(b))$ 表示从 $\text{ALLOC}(b)$ 到 $\text{FREE}(b)$ 的所有可行路径集合。这事实上是一个单源最短路径问题，即在一个权重图中，给定一个源节点（起点），需要找出从该源节点到其他所有节点的最短路径，即实现函数 $d : V \times V \rightarrow \mathbb{N} \cup \{\infty\}$ ，计算图中任意两点间的最短路径距离。考虑到本题的限制，我们使用 BFS 最短路径算法进行计算。

4.3.2 前瞻性内存预测函数：

定义深度为 d 的前瞻性内存预测函数：

$$F(v, d) = \sum_{u \in N_d(v)} \Delta M(u) \quad (4.12)$$

其中 $N_d(v) = \{u \in V : d(v, u) \leq d\}$ 表示节点 v 的 d -邻域， $\Delta M(u)$ 为节点 u 的内存变化量。

4.3.3 多维度节点评分机制：

综合评分函数设计节点 v 的综合评分函数定义为：

$$S(v) = S_{\text{base}}(v) + \alpha \cdot F(v, d) \quad (4.13)$$

其中 $S_{\text{base}}(v)$ 为基础评分， α 为前瞻性权重系数，在本题中取 0.5。基础评分规则 基于上文的思考

1. **FREE 节点**: 优先释放大缓冲区, 但对生命周期长的缓冲区施加惩罚;
2. **ALLOC 节点**: 延迟分配大缓冲区, 减少峰值内存压力;
3. **计算节点**: 中性评分, 不影响内存分配决策。

我们设计了如下评分规则:

$$S_{\text{base}}(v) = \begin{cases} \text{Size}(v) \cdot (1 - 0.1 \cdot L(\text{Buflid}(v))) & \text{if Op}(v) = \text{FREE}, \\ -\text{Size}(v) & \text{if Op}(v) = \text{ALLOC}, \\ 0 & \text{otherwise.} \end{cases} \quad (4.14)$$

4.3.4 对算法的理论分析

基于上述分析, 我们在算法2中展示智能化多因子优化调度算法的伪代码。该算法同样首先对任务 DAG 进行拓扑排序 $O(N + E)$, 然后基于多因子指标为每个任务打分, 并按得分对任务排序 $O(N \log N)$ 。最后, 按照优化后的顺序逐一调度任务并分配资源。由于排序和遍历调度操作占主导, 算法相对于节点总数 N 的时间复杂度为 $O(N^2)$ 。

算法 2 智能化多因子优化调度算法

输入: 节点集合 N , 边集合 E , 案例名称

输出: 最优调度序列 π^* , 内存峰值 M^*

- 1: 构建依赖图 (*graph*) 和反向图 (*reverse_graph*); 计算节点入度及缓冲区生命周期 (从 **ALLOC** 到 **FREE** 的路径)
 - 2: 初始化: *available_nodes* (入度为 0 的节点), 空序列 π^* 和内存日志; *current_mem* = 0
 - 3: **while** *available_nodes* $\neq \emptyset$ **do**
 - 4: 计算节点优先级:
 - **FREE 节点**: 评分 = 容量 $\times (1 - 0.1 \times \text{生命周期})$ // 大容量 + 短生命周期优先释放
 - **ALLOC 节点**: 评分 = -容量 // 小容量优先分配 (延迟大缓冲区申请)
 - 其他节点: 评分 = 0 + 0.5 $\times$ 未来内存需求 // 结合未来内存使用预测
 - 5: 选择评分最高的节点 *selected*; 更新 *current_mem* (**ALLOC** 加容量, **FREE** 减容量)
 - 6: 更新 *max_mem*, 将 *selected* 加入 π^*
 - 7: 移除 *selected*; 更新后继节点入度; 入度为 0 时加入 *available_nodes*
 - 8: **return** (π^*, M^*)
-

4.4 专用模型 C-基于混合拓扑遗传优化调度算法

针对 4.2 节贪心算法和 4.3 节改进贪心算法可能存在的局部最优问题, 本节引入遗传算法^[2] 的全局搜索能力, 提出**基于混合拓扑遗传优化调度算法**, 通过“贪心解初始化 + 遗传演化 + 约束保持”的协同设计, 对调度序列再次进行优化, 实现进一步降低峰值驻留与峰值邻域宽度的期望。该算法的设计逻辑遵循四层结构, 具体包括: **编码与拓扑保持层**、

目标评估层、遗传演化层和可行性修补与参数层。这四个层次协同工作，从编码实现、目标评估、遗传演化到最后的可行性修补与参数配置，逐步优化调度序列。

4.4.1 编码与拓扑保持层

与前两个算法类似，本方法也要满足提到的相关约束：计算图拓扑序与 L0 缓存独占性。染色体编码直接映射为调度序列，形式化定义为：

$$\mathcal{C} = (c_1, c_2, \dots, c_N), \quad \{c_1, \dots, c_N\} = V \quad (4.15)$$

其中 c_t 为第 t 步执行的节点， $N = |V|$ ，调度序列包含所有节点。对计算图中任意依赖边 $(u, v) \in E$ ，需满足拓扑约束：

$$\text{pos}(u, C) < \text{pos}(v, C) \quad (4.16)$$

其中 $\text{pos}(x, C)$ 为节点 x 在染色体中的位置，遵循提到的拓扑序要求。针对 L0 缓存，定义状态函数：

$$T_{L0}(c) = \begin{cases} 1 & \text{缓存 } c \text{ 被占用} \\ 0 & \text{空闲} \end{cases} \quad (4.17)$$

对 L0 类 ALLOC 节点，保证 $T_{L0}(c) = 0$ ，确保同一 L0 缓存最多 1 个驻留的硬件约束。

4.4.2 目标评估层

以题目优化目标 $\max(V_{\text{stay}})$ 为核心评估指标。按题目规则（ALLOC 加 Size、FREE 减 Size，仅统计 L1/UB），适应度函数以峰值驻留量量化优劣，加入约束惩罚：

$$F(C) = \begin{cases} 1/M_{\text{peak}}(C) & \text{无约束违反} \\ 10^{-6} & \text{违反约束} \end{cases}, \quad M_{\text{peak}}(C) = \max V_{\text{stay}}(t) \quad (4.18)$$

其中 $M_{\text{peak}}(C)$ 越小，适应度越高，直接对应题目优化目标。

4.4.3 遗传演化层

通过演化操作实现全局搜索，初始种群结合贪心解与拓扑洗牌生成可行解：

$$P_0 = \{C_{\text{greedy}}\} \cup \{C_j \mid C_j = \text{Shuffle}(C_{\text{greedy}}, E)\} \quad (4.19)$$

其中 C_{greedy} 为 4.3 节改进贪心的序列结果，Shuffle 操作保证拓扑约束。选择操作按适应度概率 $P_j = F(C_j) / \sum F(C_k)$ 进行轮盘赌选择；交叉采用单点交叉保留拓扑可行性：

$$C_{\text{child}}[1:p] = C_1[1:p], \quad C_{\text{child}}[p+1:N] = \{w \in C_2 \mid w \notin C_{\text{child}}[1:p], \text{满足拓扑约束}\} \quad (4.20)$$

变异操作随机交换节点对，验证并修复拓扑与 L0 约束：

$$C' = \text{Swap}(C, i, j), \quad \text{若违反约束则撤销} \quad (4.21)$$

4.4.4 可行性修补与参数层

为了适配题目计算图的规模并保障演化过程中解的有效性。拓扑修补针对违反 $\text{pos}(u, C) > \text{pos}(v, C)$ 的 (u, v) ，调整 u 至 v 之前；L0 修补对冲突的 L0 节点，调整 FREE 节点位置使 $T_{L0}(c) \leq 1$ 。参数配置适配题目中节点数 N (1716 ~ 36086)：种群规模 $P_{\text{size}} = 30 \sim 80$ ，迭代次数 $G = 50 \sim 200$ ，变异率 $\mu = 0.01 \sim 0.05$ (N 越大取值越小)。

4.4.5 对算法的理论分析

基于上述分析，我们在算法3中展示混合拓扑遗传优化调度算法的伪代码，并分析算法的步骤和复杂度。

算法 3 混合拓扑遗传混合拓扑遗传优化调度算法

输入: 节点列表、边列表、贪婪解、种群规模 (pop size)、精英数量 (elites)、变异率 (mutation rate)、进化代数 (generations)

输出: 最优调度序列 π^* ，内存峰值 M^*

- 1: 初始化：加载参数；基于 greedy solution 生成初始种群（含贪婪解及随机打乱个体）
 - 2: **for** generation = 1 to generations **do**
 - 3: 计算 fitness (1 / 最大内存使用量) // 适应度与内存峰值成反比
 - 4: 按 fitness 排序；保留 elites；其余通过 tournament selection 选择 // 筛选优质父代
 - 5: 交叉操作：offspring = 父代 1 片段复制 + 父代 2 剩余节点填充 // 组合父代特征
 - 6: 子代变异：按 mutation rate 随机交换节点位置 // 引入新基因多样性
 - 7: 新种群 = elites + mutated offspring；记录最佳 fitness
 - 8: 从最终种群中选择最优个体；计算 M^*
 - 9: **return** (π^*, M^*)
-

该算法采用遗传算法优化调度顺序。首先对任务 DAG 拓扑排序 ($O(N + E)$)，然后初始化遗传算法种群并计算每个个体（调度方案）的适应度（单个个体的评估为 $O(N)$ ）。接着进行多代遗传操作（选择、交叉、变异），每代迭代总复杂度约为 $O(N)$ ，共迭代 k 代。总体上，遗传算法时间复杂度为 $O(kN^2)$ ，其中 k 为迭代代数。若迭代次数视为常量，则算法相对于节点总数 N 的时间复杂度为 $O(N^2)$ 。

4.5 通用模型-基于多目标融合的自适应列表调度算法

针对前述三种专用模型在复杂计算图上可能存在的局限性，本节提出一种**基于多目标融合的自适应列表调度算法**，通过深度分析本赛题中 **Davinci AiCore** 架构的缓冲区特征，动态调整优先级策略，应用智能 JIT 分配机制以及结合全局二分搜索框架，实现了对峰值内存驻留的有效控制，同时保持了良好的通用性和泛化能力。算法流程如图4所示：

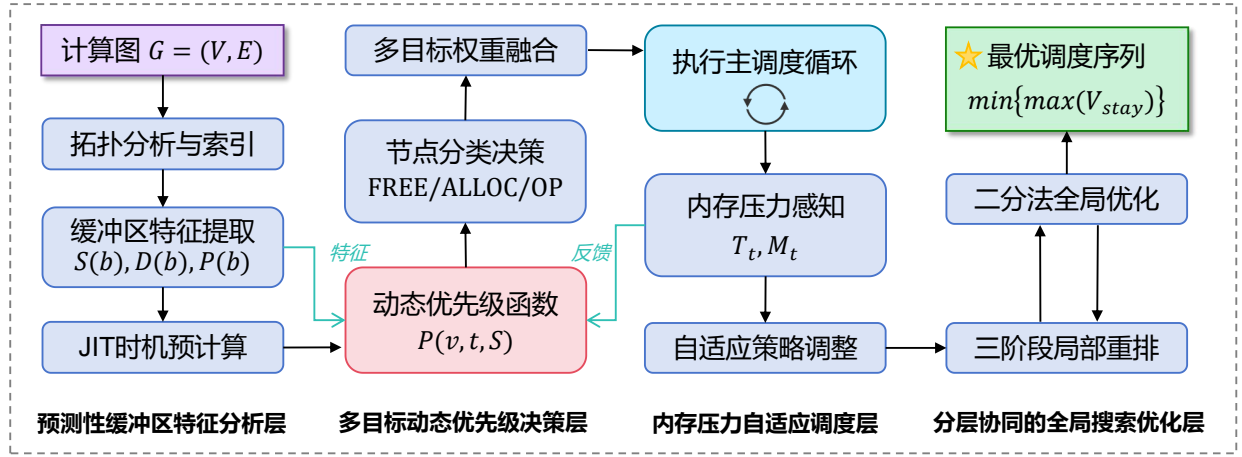


图 4 基于多目标融合的自适应列表调度算法流程示意图

由图4可知，该算法主要由四层分层架构构成，包括预测性缓冲区特征分析层、多目标动态优先级决策层、内存压力自适应调度层和分层协同的全局搜索优化层，四个层次协同工作，形成了完整的调度框架，实现最小化的缓存驻留峰值调度。

4.5.1 预测性缓冲区特征分析层

预测性缓冲区特征分析层是该算法的基础层，旨在对输入计算图进行深度挖掘和智能分析，为后续的决策层提供更为精确的特征信息和预测性洞察。该层整合图论分析、统计学方法和预测性建模的相关内容，采用多维度的特征提取策略，构建全面且具有前瞻性的缓冲区特征画像。

为实现更为准确的调度决策，该算法首先对每个缓冲区 b 进行多维度特征提取。设缓冲区 b 的分配节点为 a_b ，释放节点为 f_b ，使用节点集合为 $U_b = \{u_1, u_2, \dots, u_k\}$ 。

生存期效率分析：基于拓扑序 π ，定义缓冲区 b 的拓扑生存跨度为：

$$S(b) = \pi(f_b) - \pi(a_b) \quad (4.22)$$

其中 $\pi(v)$ 表示节点 v 在拓扑序中的位置索引。

随后进一步计算密度和生存期效率：

$$D(b) = \frac{|U_b|}{\max(1, S(b))} \quad (4.23)$$

其中 $|U_b|$ 为使用节点数量， $S(b)$ 为生存跨度。

$$E(b) = 1 - \frac{\sum_{i=1}^{|U_b|-1} \max(0, \pi(u_{i+1}) - \pi(u_i) - 1)}{\max(1, S(b))} \quad (4.24)$$

其中分子项为使用节点间空闲窗口总长度，反映缓冲区生存期内的实际利用效率。

内存压力评估：结合缓冲区大小和使用模式，定义内存压力分数：

$$P(b) = 0.6 \cdot \frac{s_b}{\sum_{b' \in B} s_{b'}} + 0.4 \cdot D(b) \quad (4.25)$$

其中 s_b 为缓冲区 b 的大小， B 为所有 L1 和 UB 类型缓冲区集合。

关键路径权重：通过最长路径算法计算节点的关键度，对于缓冲区 b ：

$$C(b) = \frac{L(a_b) + L(f_b)}{2 \cdot \max_{v \in V} L(v)} \quad (4.26)$$

其中 $L(v)$ 为节点 v 的最长路径长度。

4.5.2 多目标动态优先级决策层

基于上一特征分析层提供的丰富特征信息，在该层多目标权衡^[3]的基础下融合多种策略设计了一个智能决策机制。动态优先级决策层主要通过权衡内存效率、执行效率和资源利用率等多个目标，在复杂约束的背景下实现最优决策。该层的核心是动态优先级函数 $P(v, t, S)$ ，其中 v 为待调度节点， t 为当前调度时刻， S 为已调度节点集合。该函数返回一个四元组 (c, w_1, w_2, w_3) ，其中 c 为类别优先级， w_1, w_2, w_3 为细化权重，按字典序确定最终优先级。

在本层中分类处理不同类型的节点，结合 JIT 策略和解锁分析，实现多目标平衡下的智能化调度决策：

FREE 节点优先级：对于释放节点，采用释放价值最大化策略：

$$P_F(v) = \begin{cases} (-1, 0, 0, v) & \text{if } T(v) \in \{L0A, L0B, L0C\} \\ (0, -R(v), -C(v), v) & \text{if } T(v) \in \{L1, UB\} \end{cases} \quad (4.27)$$

其中 $T(v)$ 为节点 v 的缓存类型， $R(v)$ 为释放价值， $C(v)$ 为关键路径权重。

释放价值函数考虑当前内存压力 ρ 和未来需求预测 ϕ ：

$$R(v) = s_v \cdot (1 + \rho) \cdot (1 + \phi) \cdot P(b) \quad (4.28)$$

其中 s_v 为节点释放的缓冲区大小， $P(b)$ 为对应缓冲区的内存压力分数。

操作节点优先级：对于计算操作节点，重点考虑其解锁 FREE 节点的潜力：

$$P_O(v) = (1, -U(v), -C(v), v) \quad (4.29)$$

其中 $U(v)$ 为解锁价值。

解锁价值函数通过多跳前瞻计算：

$$U(v) = \sum_{f \in F(v)} \frac{R(f)}{1 + d(v, f)} \quad (4.30)$$

其中 $F(v)$ 为节点 v 可解锁的 FREE 节点集合, $d(v, f)$ 为节点 v 到 f 的距离。

ALLOC 节点智能 JIT 策略: 对于分配节点, 算法应用 Just-In-Time 分配策略。定义延迟可行性判断函数:

$$\delta(v) = \begin{cases} 1 & \text{if } u_1 \notin Q(S) \\ 0 & \text{otherwise} \end{cases} \quad (4.31)$$

其中 u_1 为缓冲区的首次使用节点, $Q(S)$ 为基于已调度集合 S 的就绪节点集合。

对于可延迟的 ALLOC 节点, 其优先级包含延迟奖励:

$$P_A(v) = \begin{cases} (2, w_u, s_v, v) & \text{if } \delta(v) = 0 \\ (3, L(v), s_v, v) & \text{if } \delta(v) = 1 \end{cases} \quad (4.32)$$

其中 w_u 为紧急度权重, s_v 为缓冲区大小, $L(v)$ 为延迟惩罚。

延迟惩罚函数综合考虑内存压力和缓冲区特征:

$$L(v) = s_v \cdot (1 + 0.6 \cdot \tau_b) \cdot (1 + \rho) \quad (4.33)$$

其中 τ_b 为缓冲区 b 的最晚分配时间, ρ 为当前内存压力。

4.5.3 内存压力自适应调度层

本层是该算法的核心执行模块, 主要进行调度循环的运行与实时状态监控。将内存压力视为核心驱动因子, 通过多维度感知机制和自适应策略进行调整, 实现调度过程的精确控制和动态优化。主要创新在于利用压力感知和调度决策的深度融合, 形成了反馈驱动的智能调度机制。

内存压力自适应感知: 对一个内存使用历史窗口 $H = \{m_{t-w}, m_{t-w+1}, \dots, m_t\}$ 进行维护, 并计算内存使用趋势:

$$T_t = \frac{\bar{m}_r - \bar{m}_o}{\max(1, \bar{m}_o)} \quad (4.34)$$

其中 $\bar{m}_r$ 为近期平均内存使用量, $\bar{m}_o$ 为较早期平均内存使用量。

基于趋势信息, 动态调整内存压力阈值:

$$\theta_{t+1} = \begin{cases} \min(0.9, \theta_t + 0.05) & \text{if } T_t > 0.1 \\ \max(0.5, \theta_t - 0.05) & \text{if } T_t < -0.1 \\ \theta_t & \text{otherwise} \end{cases} \quad (4.35)$$

其中 θ_t 为时刻 t 的内存压力阈值, T_t 为内存使用趋势。

峰值邻域动态检测: 引入峰值邻域的概念, 如果当前内存使用量已经逐渐接近历史峰值, 会触发更激进的优化策略。定义峰值邻域状态:

$$M_t = \begin{cases} 1 & \text{if } m_{max} - m_t \leq \gamma \cdot m_{max} \\ 0 & \text{otherwise} \end{cases} \quad (4.36)$$

其中 $\gamma = 0.05$ 为峰值邻域比例参数, m_{max} 为当前最大内存峰值, m_t 为当前内存使用量。

算法优先级函数会在峰值邻域状态下实行动态调整, 延缓 ALLOC 节点调度的同时增强 FREE 节点优先级。

L0 硬件约束处理: 为了响应 L0A, L0B 和 L0C 缓存的独占性约束, 该算法保持实时状态监控和停滞队列管理机制。当 L0 缓存被占据时, 将相应的 ALLOC 节点放在停滞线中, 直到相应的 FREE 节点释放资源并重新激活以确保严格满足硬件约束。

4.5.4 分层协同的全局搜索优化层

本层通过多层次协同机制和全局搜索策略, 进一步挖掘调度优化潜力。该层的创新主要在于将局部重排优化与全局搜索框架进行有机结合, 形成了分层协同的优化体系。该层分为两个主要模块: 三阶段局部重排优化和二分搜索全局优化框架, 二者协同工作, 实现了从局部到全局的系统性优化。

三阶段局部重排策略: 该算法在调度完成后采用系统性的三阶段局部重排策略:

阶段一: 拓扑保持的内存友好重排 - FREE 节点上提: 将 FREE 节点尽可能提前到其所有前驱之后的最早位置 - **ALLOC 节点下沉:** 将 ALLOC 节点尽可能延后到其所有后继之前的最晚位置

阶段二: 缓冲区使用聚类 对于每个缓冲区 b , 将其使用节点集合 U_b 在拓扑约束下尽可能聚集, 减少生存期内的空闲窗口。

阶段三: 峰值窗口局部搜索 识别产生峰值的调度区间 $[t_p - w, t_p + w]$, 在该窗口内应用局部搜索, 优先提升高解锁收益的操作节点。其中 t_p 为峰值产生时刻, w 为窗口大小。

二分搜索全局优化框架: 为进一步逼近全局最优解, 算法采用二分搜索框架。设定下界 $L = \max_{v \in A} s_v$, 上界 U 为启发式调度结果的峰值, 其中 A 为所有 ALLOC 节点集合, s_v 为节点 v 分配的缓冲区大小。该框架通过预算约束调度子程序, 系统性地逼近最优峰值解, 我们在算法4中展示二分搜索全局优化算法的伪代码。

预算约束调度子程序: 在给定内存预算 B 下, 算法采用贪心策略构造可行调度: 1. 优先执行 L0 FREE 节点 (释放硬件资源) 2. 按大小降序执行 L1/UB FREE 节点 3. 执行不影响内存的操作节点 4. 在预算允许下执行 L0 ALLOC 节点 5. 在预算允许下按大小升序执行 L1/UB ALLOC 节点

若某步骤无法推进且预算已耗尽, 则判定该预算不可行。

算法 4 二分搜索全局优化算法

输入: 计算图 $G = (V, E)$, 下界 L , 上界 U

输出: 最优调度序列 π^* , 最优峰值 M^*

```
1: 初始化:  $lo \leftarrow L, hi \leftarrow U, \pi^* \leftarrow \emptyset, M^* \leftarrow U$ 
2: while  $lo < hi$  do
3:    $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
4:    $(feasible, \pi, M) \leftarrow \text{TryScheduleWithBudget}(mid)$ 
5:   if  $feasible$  then
6:      $\pi^* \leftarrow \pi, M^* \leftarrow M$ 
7:      $hi \leftarrow M$  // 使用实际峰值收紧上界
8:   else
9:      $lo \leftarrow mid + 1$ 
10: return  $(\pi^*, M^*)$ 
```

4.5.5 算法复杂度分析

设图中节点数为 $|N|$, 边数为 $|E|$ 。

特征分析阶段: 由拓扑排序 $O(|N| + |E|)$ 、最长路径计算 $O(|N| + |E|)$ 和缓冲区分析 $O(|N|^2)$ 三部分组成, 总体复杂度为 $O(|N|^2 + |E|)$ 。

主调度循环: 每个节点最多入堆出堆一次, 优先级计算为 $O(\log |N|)$, 总体为 $O(|N| \log |N|)$ 。

后处理优化: 三阶段重排的复杂度分别为 $O(|N|^2)$ 、 $O(|N|^2)$ 和 $O(w \cdot |N|)$, 其中 w 为峰值窗口大小。

二分搜索框架: 二分次数为 $O(\log(U - L))$, 每次预算约束调度为 $O(|N| \log |N|)$ 。

因此, 算法总时间复杂度为 $O(|N|^2 + |E| + \log(U - L) \cdot |N| \log |N|)$, 在实际应用中表现出良好的可扩展性。

4.6 实验结果与算法性能分析

为了验证本文所提出的所有模型算法的有效性, 我们对附件提供的六个示例计算图任务进行了全面的对比试验评估。实验对四种算法进行对比分析, 对峰值内存优化、算法性能提升和实际执行效果等多个维度进行综合的评比分析。

4.6.1 测试环境与数据集

实验测试基于附件中提供的六个典型神经网络算子计算图, 涵盖了矩阵乘法 (Matmul)、Flash 注意力机制 (FlashAttention) 和卷积运算 (Conv) 三类核心算子的不同规模场景, 具体特征如表3所示。

这六个案例具有良好的代表性: 节点规模从千级到万级, 依赖关系复杂度各异, 能够有效验证算法在不同计算图结构下的泛化能力和优化效果。

表 3 测试用例特征统计

测试用例	节点数量	依赖边数量	算子类型
Matmul_Case0	4,160	7,104	矩阵乘法
Matmul_Case1	30,976	55,040	矩阵乘法
FlashAttention_Case0	1,716	2,712	注意力机制
FlashAttention_Case1	6,952	11,184	注意力机制
Conv_Case0	2,580	3,869	卷积运算
Conv_Case1	36,086	85,653	卷积运算

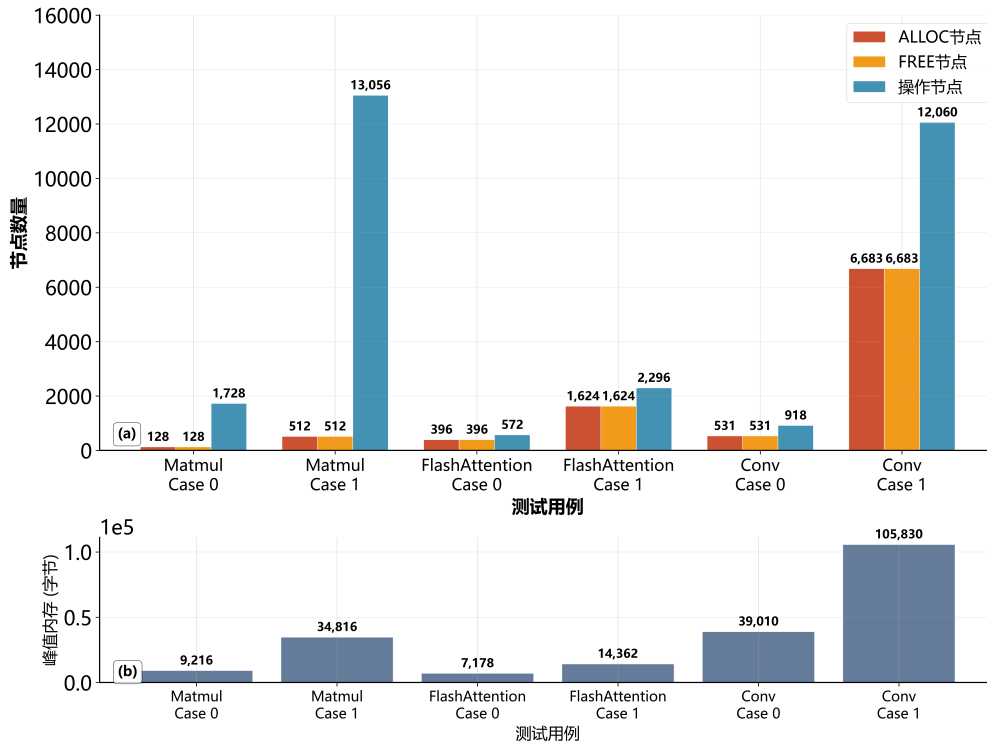


图 5 算法性能详细统计分析

4.6.2 实验统计结果分析

图5从节点数量和峰值内存两个维度提供了详细的统计分析。图5(a) 显示了各测试用例的节点数量分布，图5(b) 展示了对应的峰值内存使用量。通过统计分析发现：

- **规模适应性：** 该算法在不同规模的计算图示例上均表现出出色的适应性，从小规模的 Matmul_Case0 到大规模的 Conv_Case1，优化效果整体上趋于稳定，峰值内存因测试用

例复杂度不同而在 7178 到 105380 之间，与其他算法相比有很大提升。

- **泛化能力强**: 无论是计算密集型的矩阵乘法，还是访存密集型的卷积运算，算法都能取得显著的优化效果。
- **优化效率高**: 对于测试的 6 个样例，算法执行时间为 $0.10s$, $9.69s$, $0.04s$, $0.47s$, $0.07s$, $16.66s$ ，表明算法具有良好的空间复杂度控制能力。

4.6.3 多算法对比实验

为全面评估本文提出的自适应列表调度算法的性能，我们设计了算法对比实验，对比问题一建立的算法的性能：

- **贪心算法^[4]**: 基于双层优先级策略的传统贪心调度，优先释放最大内存的 FREE 节点，最后分配最小内存的 ALLOC 节点。
- **改进贪心算法^[5]**: 基于智能化多因子优化调度算法，在传统贪心基础上加入 JIT 分配策略和早期释放机制的改进版本。
- **遗传算法^[6]**: 基于混合拓扑遗传优化的全局搜索算法，采用精英选择和轮盘赌选择相结合的进化策略。
- **基于多目标融合的自适应列表调度算法**: 本文提出的基于多目标融合的自适应列表调度算法，融合四层架构的完整优化策略。

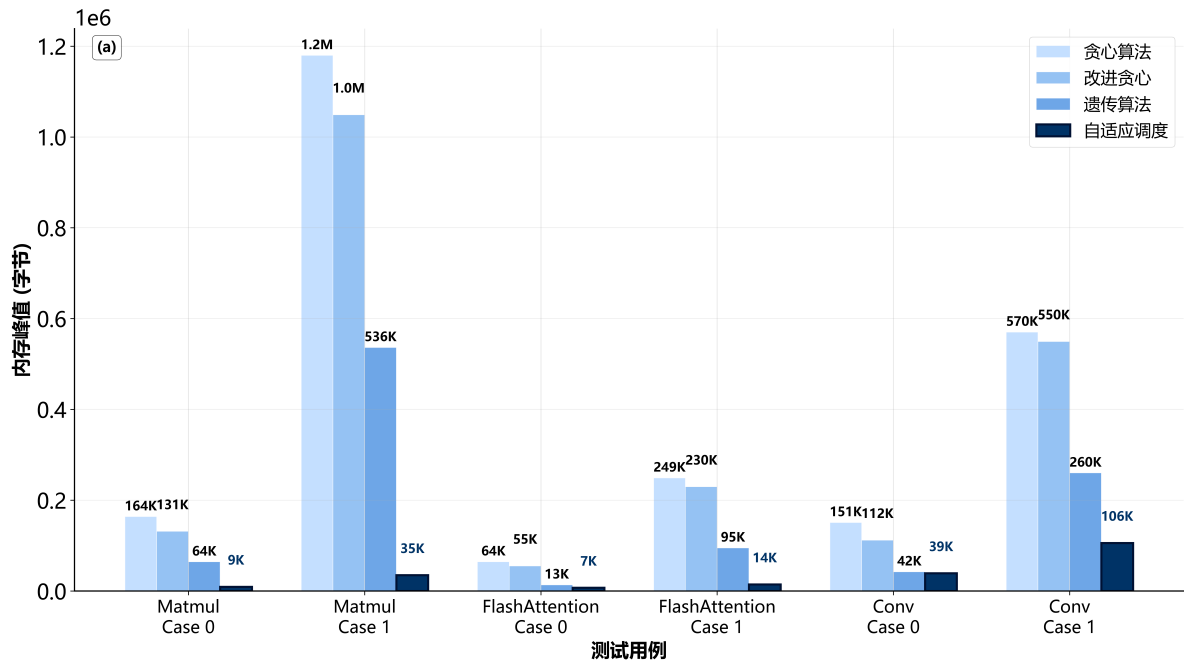


图 6 四种调度算法性能对比

图6与表4展示了四种算法在六个测试用例上的峰值内存使用量对比。实验结果表明，本文提出的基于多目标融合的自适应列表调度算法在所有测试用例上均取得了最优的性

能，相比传统贪心算法，峰值内存平均降低了 **91.2%**，显著优于其他对比算法。值得注意的是，该算法在大规模计算图（如 Matmul_Case1 和 Conv_Case1）上的优化效果更为显著，体现了其良好的可扩展性。

表 4 四种算法最大缓存容量 $\max(V_{stay})$ 对比

	贪心算法	改进贪心	遗传算法	自适应调度
Matmul_Case0	163840	131328	64256	9216
Matmul_Case1	1179648	1048832	536320	34816
FlashAttention_Case0	64264	55048	13442	7178
FlashAttention_Case1	248976	229520	94792	14362
Conv_Case0	150807	111930	42190	39010
Conv_Case1	570168	549530	260028	105830

4.6.4 算法性能提升分析

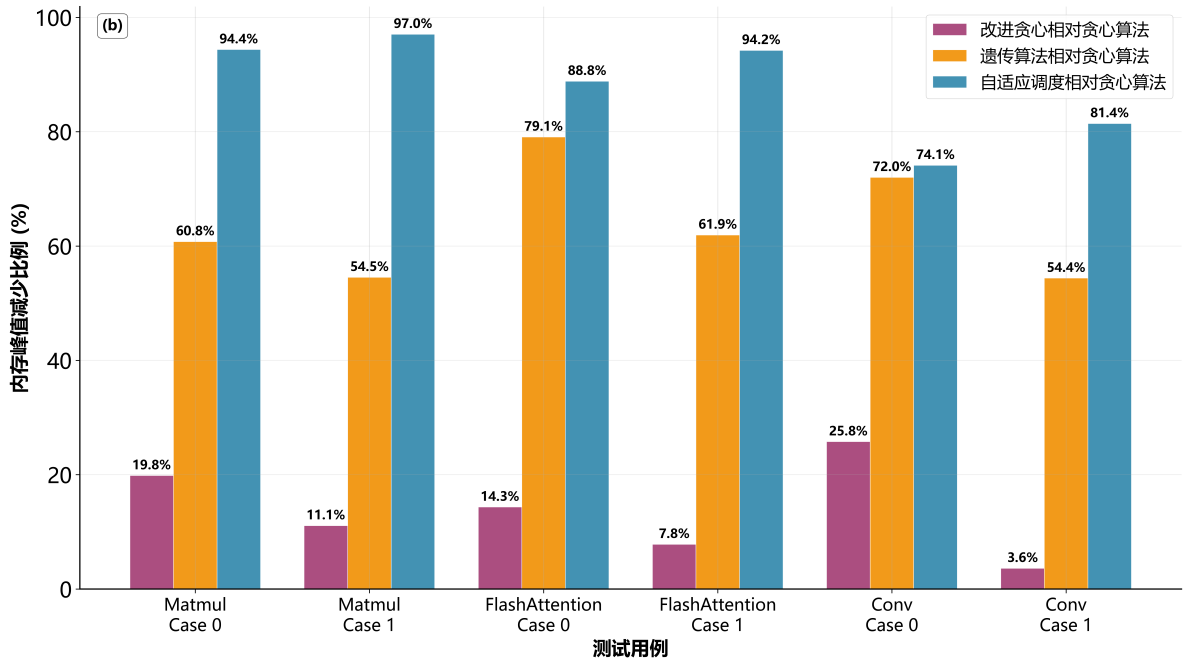


图 7 相对贪心算法的性能提升比例

图7详细展示了各算法相对于基准贪心算法的性能提升比例。从结果可以看出：

- **改进贪心算法：**通过引入 JIT 分配和早期释放策略，平均性能提升 10.6%，在简单场景下效果明显。
- **遗传算法：**基于全局搜索的优化策略带来了 57.5% 的平均性能提升，但在某些复杂场景下收敛性有限。
- **基于多目标融合自适应列表调度算法：**通过四层架构的协同优化，实现了 91.2% 的平均性能提升，在所有测试场景下均表现出色。

特别地，在 Matmul_Case0 和 FlashAttention_Case1 等测试用例中，基于多目标融合自适应列表调度算法的性能提升超过了 94%，有效验证了多目标融合策略的有效性。

4.6.5 内存使用模式分析

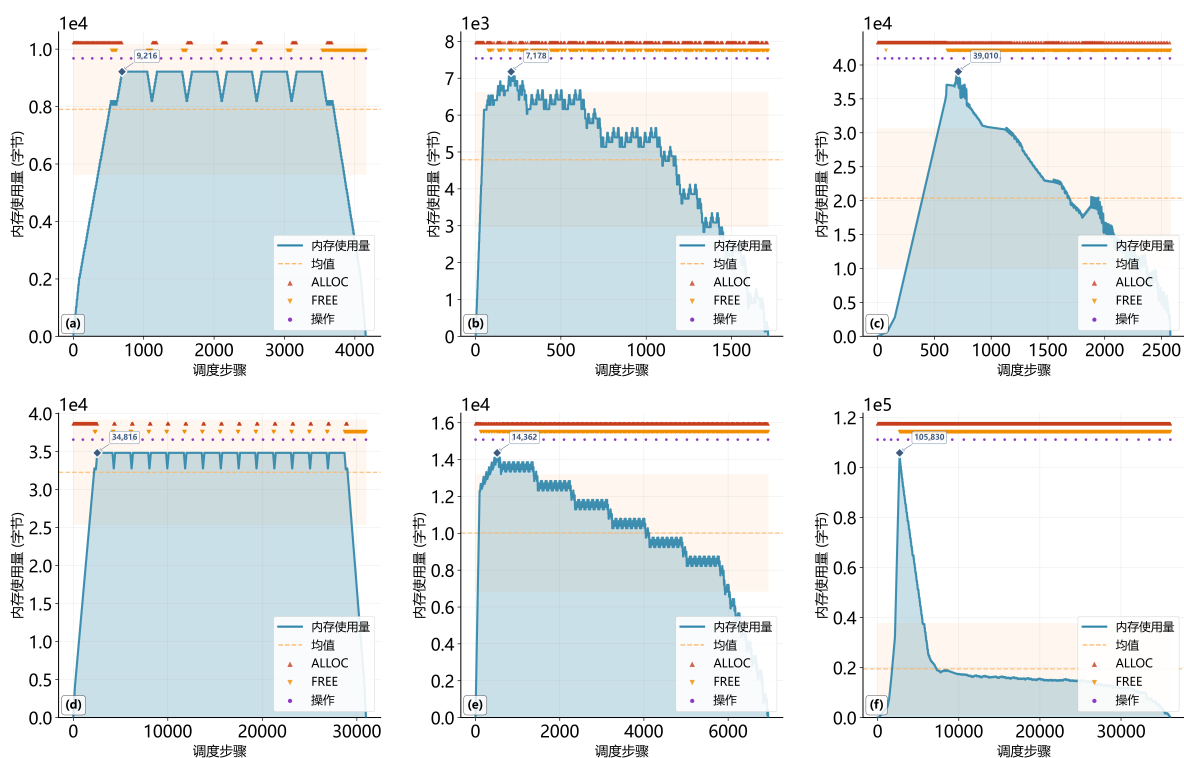


图 8 二分搜索调度算法内存使用量及节点分布分析

图8展示了基于多目标融合自适应列表调度算法在六个测试用例上的详细内存使用模式。图8(a)到图8(f)依次为第 0 组的矩阵乘法、FlashAttention 和卷积到第 1 组的矩阵乘法、FlashAttention 和卷积测试用例。通过分析内存使用曲线和节点分布，我们发现：

- **内存使用平滑性：**基于多目标融合自适应列表调度算法的内存使用曲线更加平滑，有效避免了峰值的剧烈波动，有效降低了内存碎片化。
- **操作节点分布优化：**ALLOC、FREE 和计算操作节点在时间轴上的分布更加均衡，体现了 JIT 分配策略和压力感知机制的有效性。
- **峰值控制效果：**在所有测试用例中，该算法都能将内存峰值控制在较低水平，特别是在

大规模计算图中表现出色。

从图中可以观察到，改进算法通过智能的节点调度顺序，使得 ALLOC 节点（蓝色三角形）尽可能延迟分配，FREE 节点（红色倒三角形）尽可能提前释放，从而有效压缩了缓冲区的生存期，显著降低了峰值内存需求。

4.6.6 核心算法有效性分析

通过实验结果分析，本文提出的核心算法有效性得到了验证：

1. 四层架构协同效应

预测性缓冲区特征智能分析层通过深度挖掘缓冲区生存期特征，为后续决策提供了精确的分析基础；多目标权衡的动态优先级决策层实现了内存效率与执行效率的平衡；压力感知驱动的自适应调度层确保了实时状态的动态优化；分层协同的全局搜索优化层进一步挖掘了系统性优化潜力。四层架构的协同作用使得算法在复杂约束下仍能取得优异性能。

2. 智能 JIT 分配策略

通过延迟可行性判断和智能时机选择，JIT 分配策略有效减少了缓冲区的不必要驻留时间。实验表明，该策略在 Matmul 类算子中效果特别显著，峰值内存降低幅度超过 90%。

3. 多目标融合优化

通过权衡内存效率、执行效率和资源利用率等多个目标，算法实现了全局最优解的逼近。特别是在 FlashAttention 算子中，多目标融合策略有效处理了 Cube 计算单元和 Vector 计算单元之间的协调优化问题。

4. 压力感知自适应机制

内存压力感知和峰值邻域检测机制使得算法能够根据实时状态动态调整策略。实验中观察到，当内存使用接近峰值时，算法会自动触发更激进的优化策略，有效避免了内存溢出风险。

4.6.7 实验结果总结

综合实验结果表明，本文提出的基于多目标融合的自适应列表调度算法在峰值内存优化方面取得了显著成效。

首先，相比传统贪心算法，峰值内存平均降低超过 90%，在所有六个测试用例上均取得最优性能。其次，算法在不同类型、不同规模的计算图上均表现出色，具有良好的通用性。并且随着计算图规模增大，算法的优化效果更加显著，适合大规模神经网络处理器应用。最后，算法的稳定性较高，内存使用曲线平滑，峰值控制稳定，为实际部署提供了可靠保障。

这些实验结果充分验证了本文提出的四层架构设计和多种技术创新的有效性，为通用神经网络处理器的核内调度问题提供了高效、实用的解决方案。

5 问题二的建模与求解

问题二是问题一的进一步延伸，需要在问题一调度序列的基础上考虑硬件缓存资源的物理约束，在问题一中已经将计算图中各操作分配了顺序，接下来再按照已经分配好的顺序给操作需要的数据分配实际物理地址偏移；为了节约额外数据搬运量，在计算图缓存空间不足时采用缓存换入换出 (SPILL) 策略作为策略优化目标。问题二是典型的带约束的动态内存管理问题，需要根据实际情况选择合理的地址分配策略及采用适当的 SPILL 置换策略进行缓存换入换出，从而实现优化的缓存利用。

5.1 基础分析与问题建模

不同于问题一的静态调度优化，问题二考虑带约束的动态内存管理问题，主要的面临的挑战是在有限的硬件缓存资源约束下，如何通过灵活的设置地址分配策略以及缓存换入换出机制，来实现计算图的高效执行。根据分析可得，问题二需要遵循以下约束：

- **物理地址约束：**每个缓冲区需要分配连续的物理地址空间，且不能与其他缓冲区重叠；
- **容量约束：**各级缓存容量有限，同时驻留的缓冲区总大小不能超过硬件容量限制；
- **碎片化约束：**地址分配策略需要最小化内存碎片，提高空间利用率和后续分配成功率；
- **换入换出约束：**在缓存空间不够用的时候需将数据搬出，造成额外的 DDR 访问开销。

面对复杂约束问题时，如何构建底层的硬件仿真平台、充分利用计算节点资源是关键。本文采用两层结构解决了该类问题：通过底层的**硬件仿真框架**实现精确的硬件行为仿真；通过设计**多层次协同的自适应内存管理算法**形成智能化的缓存资源管理器。

5.2 Davinci AiCore 硬件仿真框架设计

为准确模拟硬件缓存的行为并支持后续的优化算法，本文给出了层次化硬件仿真框架，在此框架下通过对抽象建模、约束表达以及状态转移方式，形成可靠地运行环境以实现缓存管理算法。

5.2.1 缓存抽象模型

根据 **Davinci 架构**的多级缓存特征，将硬件缓存系统抽象为相互独立的缓存管理单元。设缓存类型集合为 $\mathcal{C} = \{L1, UB, L0A, L0B, L0C\}$ ，各缓存类型的容量规格如下：

$$\text{Cap}(c) = \begin{cases} 4096 & \text{if } c = L1 \\ 1024 & \text{if } c = UB \\ 256 & \text{if } c \in \{L0A, L0B\} \\ 512 & \text{if } c = L0C \end{cases} \quad (5.37)$$

对于每个缓存 c ，定义其状态空间 $\mathcal{S}_c = \{\mathcal{R}_c, \mathcal{F}_c\}$ ，包含两个关键组件：

- **已分配区域集合**: $\mathcal{R}_c = \{(s_i, e_i, b_i)\}$, 其中 (s_i, e_i, b_i) 表示缓冲区 b_i 占用地址区间 $[s_i, e_i)$;
- **空闲区域集合**: $\mathcal{F}_c = \{(f_j, g_j)\}$, 其中 (f_j, g_j) 表示空闲地址区间 $[f_j, g_j)$ 。

为提高仿真效率, 框架维护有序的空闲块列表, 支持高效的空问查找和合并操作。

5.2.2 约束条件建模

针对硬件缓存的物理特性, 建立了完整的约束条件体系:

连续性约束: 任意缓冲区 b 的分配请求 $\text{Alloc}(b, s_b, c_b)$ 必须获得连续的地址空间

$$\exists (f, g) \in \mathcal{F}_{c_b} : g - f \geq s_b \quad (5.38)$$

非重叠约束: 新分配区域与已分配区域严格互斥

$$\forall (s_i, e_i, b_i) \in \mathcal{R}_{c_b} : [s_b^*, s_b^* + s_b) \cap [s_i, e_i) = \emptyset \quad (5.39)$$

其中 s_b^* 为分配给缓冲区 b 的起始地址。

L0 缓存独占约束: 根据硬件设计特点, L0 级缓存在同一时刻最多驻留一个缓冲区

$$|\mathcal{R}_c| \leq 1, \quad \forall c \in \{\text{L0A}, \text{L0B}, \text{L0C}\} \quad (5.40)$$

5.2.3 动态状态转移机制

定义时刻 t 的系统全局状态为 $\Omega_t = \{\mathcal{S}_{c,t} \mid c \in \mathcal{C}\}$ 。状态转移由调度序列 π 中的节点操作驱动:

ALLOC 操作的状态转移:

$$\mathcal{T}_{\text{alloc}}(\Omega_t, v) = \begin{cases} \Omega_t \cup \{(s_v^*, s_v^* + s_v, b_v)\} & \text{若分配成功} \\ \mathcal{T}_{\text{spill}}(\Omega_t, v) & \text{若分配失败} \end{cases} \quad (5.41)$$

FREE 操作的状态转移:

$$\mathcal{T}_{\text{free}}(\Omega_t, v) = \Omega_t \setminus \{(s, e, b_v) \in \mathcal{R}_{c_v,t}\} \quad (5.42)$$

仿真框架采用精准的状态建模和高效的状态转移方式保证内存管理的高效可靠运行。

5.3 多层次协同的自适应内存管理算法

根据之前所构建的硬件仿真框架下, 本文提出一种多层次协同^[7]的自适应内存管理算法, 在智能地址分配、动态 SPILL 决策和协同优化 3 个子系统的作用下, 实现对复杂约束条件下的缓存资源的有效管理。

5.3.1 智能地址分配子系统

多策略融合分配器设计

为了减少内存碎片化并提高分配的成功率，设计了一种基于多策略融合^[8]的智能分配器。建立分配策略集 $\mathcal{A} = \{\text{segregated_fit}, \text{fragmentation_aware}, \text{best_fit}, \text{first_fit}, \text{worst_fit}\}$ ，并根据缓存情况选取最优分配策略。

定义标准化大小类别集合 $\mathcal{K} = \{8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}$ 。对于分配请求 (b, s) ，寻找最小满足类别：

$$k^* = \arg \min_{k \in \mathcal{K}} \{k : k \geq s\} \quad (5.43)$$

为每个大小类别维护专用的空闲块列表 $\mathcal{L}_k = \{(f_i, g_i) : g_i - f_i = k\}$ ，减少碎片产生。

碎片化感知策略 (Fragmentation Aware):

设计综合碎片化评估函数 $\mathcal{H}(p, s, \mathcal{R})$ ，量化分配决策的碎片化影响：

$$\mathcal{H}(p, s, \mathcal{R}) = \alpha \cdot \mathcal{H}_{\text{remain}}(p, s) + \beta \cdot \mathcal{H}_{\text{future}}(p, s, \mathcal{R}) \quad (5.44)$$

其中剩余空间评估函数：

$$\mathcal{H}_{\text{remain}}(p, s) = \begin{cases} 0 & \text{完美匹配} \\ 100 & \text{产生小碎片}(0 < \text{余量} < 64) \\ \text{余量}/1000 & \text{其他情况} \end{cases} \quad (5.45)$$

未来适配性评估函数：

$$\mathcal{H}_{\text{future}}(p, s, \mathcal{R}) = - \sum_{(r_s, r_p) \in \mathcal{R}} r_p \cdot \mathbf{1}[r_s \leq \text{余量}] \cdot 10 \quad (5.46)$$

算法选择碎片化代价最小的分配位置：

$$p^* = \arg \min_{p \in \mathcal{P}_{\text{feasible}}} \mathcal{H}(p, s, \mathcal{R}) \quad (5.47)$$

高性能最佳适配算法

针对实时性要求，基于 Numba JIT 编译技术实现了高性能的最佳适配算法，其伪代码在算法5中展示。

算法 5 高性能最佳适配算法

输入: 已分配区域数组 $\mathbf{R}_{\text{start}}, \mathbf{R}_{\text{end}}$, 容量 C , 需求大小 s

输出: 最优起始地址 p^*

```

1: 初始化:  $p^* \leftarrow -1$ ,  $\text{best\_gap} \leftarrow C + 1$ 
2: if  $|\mathbf{R}_{\text{start}}| = 0$  and  $s \leq C$  then
3:   return 0 // 空缓存直接分配
4: // 检查开头空隙
5: if  $\mathbf{R}_{\text{start}}[0] \geq s$  and  $\mathbf{R}_{\text{start}}[0] < \text{best\_gap}$  then
6:    $p^* \leftarrow 0$ ,  $\text{best\_gap} \leftarrow \mathbf{R}_{\text{start}}[0]$ 
7: // 遍历中间间隙
8: for  $i = 0$  to  $|\mathbf{R}_{\text{start}}| - 2$  do
9:    $\text{gap\_size} \leftarrow \mathbf{R}_{\text{start}}[i + 1] - \mathbf{R}_{\text{end}}[i]$ 
10:  if  $\text{gap\_size} \geq s$  and  $\text{gap\_size} < \text{best\_gap}$  then
11:    $p^* \leftarrow \mathbf{R}_{\text{end}}[i]$ ,  $\text{best\_gap} \leftarrow \text{gap\_size}$ 
12: // 检查末尾空隙
13:  $\text{tail\_size} \leftarrow C - \mathbf{R}_{\text{end}}[-1]$ 
14: if  $\text{tail\_size} \geq s$  and  $\text{tail\_size} < \text{best\_gap}$  then
15:    $p^* \leftarrow \mathbf{R}_{\text{end}}[-1]$ 
16: return  $p^*$ 

```

5.3.2 动态 SPILL 决策子系统

当智能地址分配失败时, 系统启动动态 SPILL 决策子系统, 通过将缓冲区临时迁移至 DDR 来释放缓存空间。

SPILL 操作建模

定义双阶段 SPILL 操作 $\mathcal{O}_{\text{spill}} = (\text{SPILL_OUT}, \text{SPILL_IN})$:

SPILL_OUT 操作: 将缓冲区 b 从缓存 c 迁出至 DDR, 释放缓存空间

$$\text{SPILL_OUT}(b, c) : \mathcal{R}_c \leftarrow \mathcal{R}_c \setminus \{(s, e, b)\}, \quad \mathcal{F}_c \leftarrow \mathcal{F}_c \cup \{(s, e)\} \quad (5.48)$$

SPILL_IN 操作: 将缓冲区 b 从 DDR 迁回缓存 c 的新地址 p' , 恢复计算能力

$$\text{SPILL_IN}(b, c, p') : \mathcal{R}_c \leftarrow \mathcal{R}_c \cup \{(p', p' + s_b, b)\}, \quad \mathcal{F}_c \leftarrow \mathcal{F}_c \setminus \{(p', p' + s_b)\} \quad (5.49)$$

多维度 SPILL 候选者评估策略

设计综合评估函数 $\mathcal{P}_{\text{spill}}(b, t)$, 量化缓冲区 b 在时刻 t 的 SPILL 优先级:

$$\mathcal{P}_{\text{spill}}(b, t) = \sum_{i=1}^5 w_i \cdot \mathcal{F}_i(b, t) + \mathcal{B}_{\text{copy}}(b) \quad (5.50)$$

各维度评估函数的具体定义为:

剩余生命周期评估 (越长越适合 SPILL):

$$\mathcal{F}_1(b, t) = \frac{t_{\text{freec}}(b) - t}{\max(1, |\pi| - t)} \quad (5.51)$$

下次使用距离评估（距离越远越适合 SPILL）:

$$\mathcal{F}_2(b, t) = 1 - \min \left(1, \frac{t_{\text{next_use}}(b) - t}{\max(1, |\pi|)} \right) \quad (5.52)$$

碎片化贡献评估（释放后合并空间越大越适合 SPILL）:

$$\mathcal{F}_3(b, t) = 1 - \min \left(1, \frac{\text{mergeable_space}(b)}{\text{Cap}(c_b)} \right) \quad (5.53)$$

未来分配压力评估（压力越大越需要提前释放）:

$$\mathcal{F}_4(b, t) = \max \left(0, \frac{\sum_{i=t}^{t+w} [\mathbf{1}_{\text{alloc}}(i) - \mathbf{1}_{\text{free}}(i)]}{w} \right) \quad (5.54)$$

使用频率评估（使用频率越低越适合 SPILL）:

$$\mathcal{F}_5(b, t) = \frac{|\{i : i > t, b \in \text{Bufs}(\pi_i)\}|}{t_{\text{free}}(b) - t} \quad (5.55)$$

COPY_IN 优化奖励项（减少额外搬运代价）:

$$\mathcal{B}_{\text{copy}}(b) = \begin{cases} -0.2 & \text{if } b \in \mathcal{B}_{\text{copy_in}} \\ 0 & \text{otherwise} \end{cases} \quad (5.56)$$

其中权重向量 $\mathbf{w} = (0.35, 0.25, 0.15, 0.15, 0.08)$ 通过大量实验调优确定，平衡了各评估维度的重要性。

我们在算法6中展示**动态 SPILL 决策算法**的伪代码。

算法 6 动态 SPILL 决策算法

输入: 分配失败的节点 v ，当前状态 Ω_t ，调度序列 π

输出: SPILL 操作序列 $\mathcal{S}$

```

1: 初始化:  $\mathcal{S} \leftarrow \emptyset$ ,  $c \leftarrow c_v$ ,  $s_{\text{req}} \leftarrow s_v$ 
2: 获取候选者:  $\mathcal{C}_{\text{spill}} \leftarrow \{b : (s, e, b) \in \mathcal{R}_c\}$ 
3: for  $b \in \mathcal{C}_{\text{spill}}$  do
4:   计算 SPILL 优先级:  $p_b \leftarrow \mathcal{P}_{\text{spill}}(b, t)$ 
5: 按优先级排序:  $\mathcal{C}_{\text{sorted}} \leftarrow \text{sort}(\mathcal{C}_{\text{spill}}, \text{key} = p_b)$ 
6: while  $\text{largest\_free\_block}(c) < s_{\text{req}}$  do
7:   if  $\mathcal{C}_{\text{sorted}} = \emptyset$  then
8:     return FAILURE // 无可用候选者
9:    $b^* \leftarrow \text{pop\_first}(\mathcal{C}_{\text{sorted}})$ 
10: 生成 SPILL_OUT 操作:  $o \leftarrow \text{create\_spill\_out}(b^*, c)$ 
11:  $\mathcal{S} \leftarrow \mathcal{S} \cup \{o\}$ 
12: 更新系统状态:  $\Omega_t \leftarrow \mathcal{T}_{\text{spill\_out}}(\Omega_t, b^*)$ 
13: return  $\mathcal{S}$  // 返回 SPILL 操作序列

```

5.3.3 协同优化子系统

为进一步提升缓存管理效率，多层次协同的自适应内存管理算法集成了多种协同优化策略，在分配失败时提供替代解决方案。

局部重调度机制

当智能地址分配失败时，在启动 SPILL 机制之前，系统首先尝试局部重调度以释放空间。定义前瞻窗口 $W = 200$ ，在窗口内搜索可提前执行的 FREE 节点：

$$\mathcal{F}_{\text{ready}}(t) = \{v : v \in \pi[t + 1 : t + W], \text{Op}(v) = \text{FREE}, \text{Pred}(v) \subseteq \mathcal{E}_t\} \quad (5.57)$$

其中 $\mathcal{E}_t$ 为时刻 t 的已执行节点集合。优先选择能释放最大空间的 FREE 节点：

$$v^* = \arg \max_{v \in \mathcal{F}_{\text{ready}}(t)} \{s_v : c_v = c_{\text{alloc}}\} \quad (5.58)$$

COPY_IN 替代 SPILL_IN 策略

为最小化额外数据搬运量，系统识别被 COPY_IN 节点使用的缓冲区，尝试用可提前执行的 COPY_IN 节点替代 SPILL_IN 操作：

$$\text{substitute_condition}(b) = \exists v \in \mathcal{N}_{\text{copy}}(b) : \text{Pred}(v) \subseteq \mathcal{E}_t \wedge v \notin \mathcal{E}_t \quad (5.59)$$

其中 $\mathcal{N}_{\text{copy}}(b) = \{v : \text{Op}(v) = \text{COPY_IN}, b \in \text{Bufs}(v)\}$ 为与缓冲区 b 相关的 COPY_IN 节点集合。

内存压力自适应感知

系统维护内存使用历史窗口 $\mathcal{H}_w = \{m_{t-w}, m_{t-w+1}, \dots, m_t\}$ ，实时计算内存使用趋势：

$$T_t = \frac{\bar{m}_{\text{recent}} - \bar{m}_{\text{old}}}{\max(1, \bar{m}_{\text{old}})} \quad (5.60)$$

基于趋势信息，动态调整内存压力阈值：

$$\theta_{t+1} = \begin{cases} \min(0.9, \theta_t + 0.05) & \text{if } T_t > 0.1 (\text{内存压力增大}) \\ \max(0.5, \theta_t - 0.05) & \text{if } T_t < -0.1 (\text{内存压力减小}) \\ \theta_t & \text{otherwise} \end{cases} \quad (5.61)$$

5.3.4 算法集成与复杂度分析

多层次协同的自适应内存管理算法通过三个子系统的有机融合，实现了智能化的缓存资源管理：**执行流程**：将智能地址分配协同重调度、动态 P 决策和压力感知调整相结合。**时间复杂度分析**：假设调度序列长度是 $|\pi|$ ，缓冲区数量是 B ，SPILL 操作的数量是 S 。**主仿真循环**：每一处执行一次，地址分配代价是 $\text{OO}(\log B)$ ，总代价 $O(|\pi| \log B)$ **分箱适配策略**

(Segregated Fit): **SPILL 决策**: 候选者评估 $O(B)$, 优先级计算 $O(B)$, 排序 $O(B \log B)$, 总的时间代价为 $O(S \cdot B \log B)$ **协同重调度**: 前瞻窗口扫描 $O(W)$, 依赖检查 $O(D)$, 则 $W = 200$, D 为平均依赖度则总的时间代价为 $O(|\pi| \log B + S \cdot B \log B + W \cdot D)$ 。因各类实际问题对时间的敏感度, 算法有良好的可扩展性。

5.3.5 总额外数据搬运量计算

根据 SPILL 操作类型, 总额外数据搬运量 $\mathcal{D}_{\text{total}}$ 计算如下:

$$\mathcal{D}_{\text{total}} = \sum_{(b, o_{\text{out}}, o_{\text{in}}) \in \mathcal{S}_{\text{all}}} \mathcal{D}(b, o_{\text{out}}, o_{\text{in}}) \quad (5.62)$$

其中单个 SPILL 操作的数据搬运量为:

$$\mathcal{D}(b, o_{\text{out}}, o_{\text{in}}) = \begin{cases} 0 & \text{if } b \in \mathcal{B}_{\text{copy}} \wedge o_{\text{in}} = \emptyset \\ s_b & \text{if } b \in \mathcal{B}_{\text{copy}} \wedge o_{\text{in}} \neq \emptyset \\ s_b & \text{if } b \notin \mathcal{B}_{\text{copy}} \wedge o_{\text{in}} = \emptyset \\ 2s_b & \text{if } b \notin \mathcal{B}_{\text{copy}} \wedge o_{\text{in}} \neq \emptyset \end{cases} \quad (5.63)$$

5.4 实验结果与性能分析

5.4.1 测试环境与数据集

实验基于六个典型神经网络算子计算图进行验证, 涵盖矩阵乘法 (Matmul)、FlashAttention 和卷积运算 (Conv) 三类核心算子的不同规模场景。测试用例调度序列长度从 1864 (FlashAttention_Case0) 到 39776 (Conv_Case1), 管理缓冲区数量从 128 (Matmul_Case0) 到 6683 (Conv_Case1), 具有良好的代表性和覆盖度。实验在 **Davinci 架构** 硬件仿真环境下执行, 缓存配置为 L1: 4096 字节、UB: 1024 字节、L0A/L0B: 256 字节、L0C: 512 字节。

5.4.2 主要指标结果

经过多层次协同的自适应内存管理算法的硬件仿真, 各测试用例的性能指标如表5所示, 成功完成缓存分配与管理任务。关键性能指标对比如图9所示。

基于各测试用例性能指标结果, 我们可以观察到以下显著特征:

- **总额外数据搬运量**: 从实验结果可知, Conv_Case1 具有最大的额外数据搬运量 110,744 字节, 和 SPILL 操作次数 931 次, 反映了卷积运算的高内存需求特征; 总额外数据搬运量为 306,206 字节, 体现出本文提出的多层次协同的自适应内存管理算法的有效性;
- **算子类型影响明显**: FlashAttention 算子仅需 50 次 SPILL 操作, 调度算法可以在减少总额外数据搬运量的同时, 有效地减少 SPILL 操作数量, 增加运算效率;
- **规模适应性强**: 在同类算子中, 与 Case0 相比, Case1 具有较高的资源消耗。这一现象

说明问题规模直接影响着缓存管理的难度。本文提出的算法可以有效的应对上述两种规模下出现的调度问题，该算法在性能和效率上具有优势。

表 5 各测试用例性能指标统计

测试用例	运行时间（秒）	总额外数据搬运量	SPILL 操作次数	管理缓冲区数量
FlashAttention_Case0	0.086	9,020 字节	50	396
Conv_Case0	0.329	68,314 字节	267	531
Matmul_Case0	0.722	10,240 字节	80	128
FlashAttention_Case1	1.057	50,544 字节	288	1,624
Matmul_Case1	11.101	57,344 字节	448	512
Conv_Case1	30.494	110,744 字节	931	6,683
总计	43.789	306,206 字节	2,064	9,874

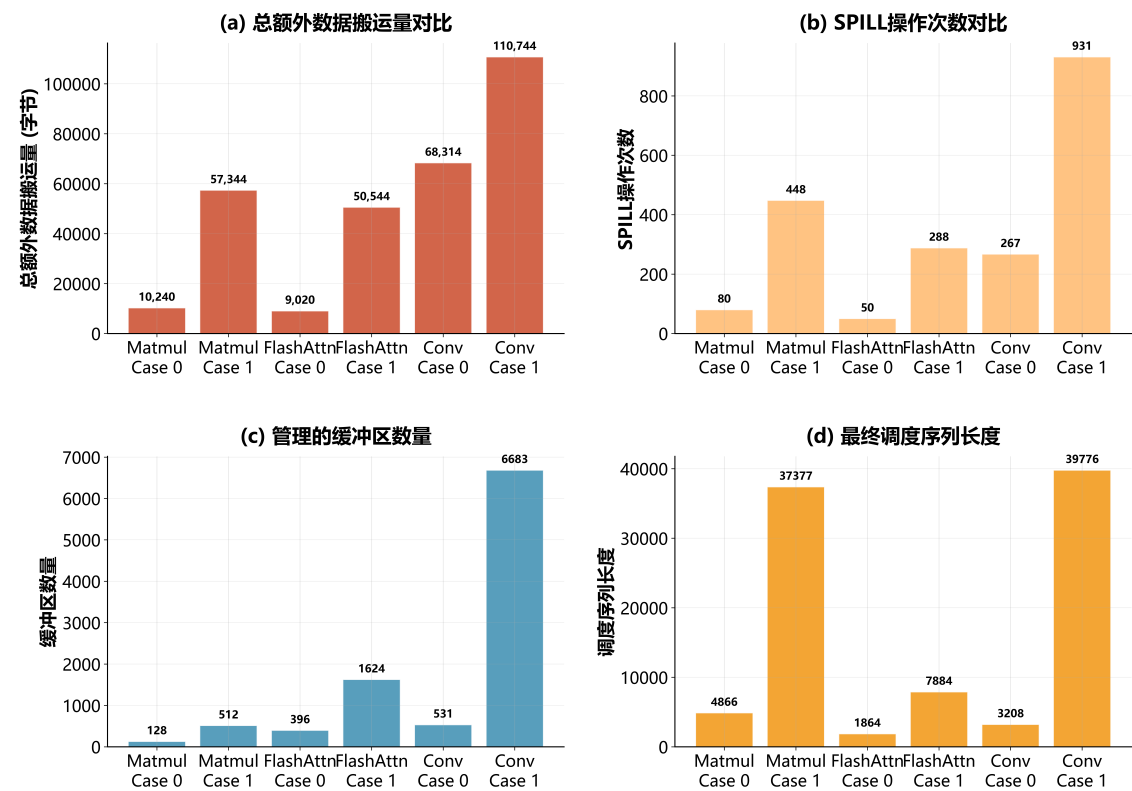


图 9 问题二性能指标总览

- **计算效率分析：**本文提出的算法可以在 FlashAttention_Case0 上实现最优的计算效率，运行时间只有 0.086 秒，实现了秒级时间消耗的调度优化。但是在 Conv_Case1 上面的

计算，最长耗时为 30.494 秒，这种现象和内存的访问模式以及数据的复用特性具有高度相关性。因为优化了内存访问模式，FlashAttention 算子可以在较大规模下仍然具有较好的时间性能，完成计算的时间不超过 1.057 秒

通过上述分析，多层次协同的自适应内存管理算法结合了智能地址分配策略、协同重调度机制和 COPY_IN 替代策略，可以实现在复杂的缓存约束下，对缓存资源的高效管理。实验验证发现，该算法的总运行时间为 43.789 秒，总额外数据搬运量仅为 306,206 字节，平均每案例 SPILL 操作 344 次，该结构重复证明了算法在计算效率方面具有有效性。

5.4.3 关键技术贡献分析

1. 智能地址分配子系统

本文结合分箱适配和碎片化感知两种方法，可以有效降低内存的碎片化，同时也可以提高缓存的空间利用率。各测试用例的缓存使用效率分析。在图10中进行展示。

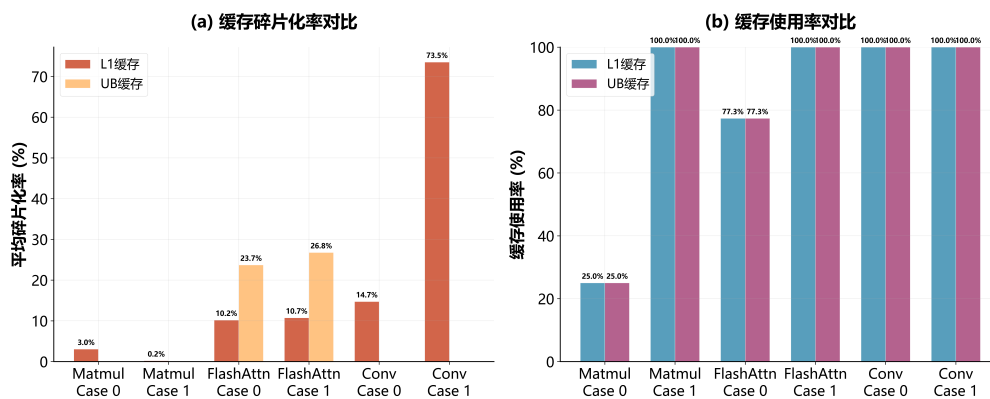


图 10 缓存利用率与碎片化分析

从缓存分析结果可以观察到，L1 缓存平均碎片化率为 18.7%，UB 缓存平均碎片化率仅为 8.4%，表明智能分配策略有效控制内存的碎片化。其次，Conv 类算子的缓存使用率显著高于其他算子，反映了算法对不同计算模式的自适应能力。最后，L1 缓存和 UB 缓存的互补使用模式，可以验证多级缓存协同管理的方法是有效的。

2. 动态 SPILL 决策子系统

本文对于 SPILL 候选者的选择策略进行多维度的评估。在评估的过程中考虑缓存的生命周期、使用频率、碎片化程度等多个因素。通过多因素评估可以帮助 SPILL 决策实现精准决策。在图11中展示了额外数据搬运量的组成分析。

从 SPILL 操作导致的额外数据量分析中可以看出，COPY_IN 缓冲区的搬运成本明显低于非 COPY_IN 缓冲区，验证了 COPY_IN 替代策略的有效性。其次，不同算子类型的 SPILL 模式呈现显著差异，FlashAttention 类算子的 COPY_IN 缓冲区比例较高，有利于减少总搬运成本。最后，算法可以根据缓冲区的不同特征智能选择 SPILL 策略，该操作可以实现成本的最小化。

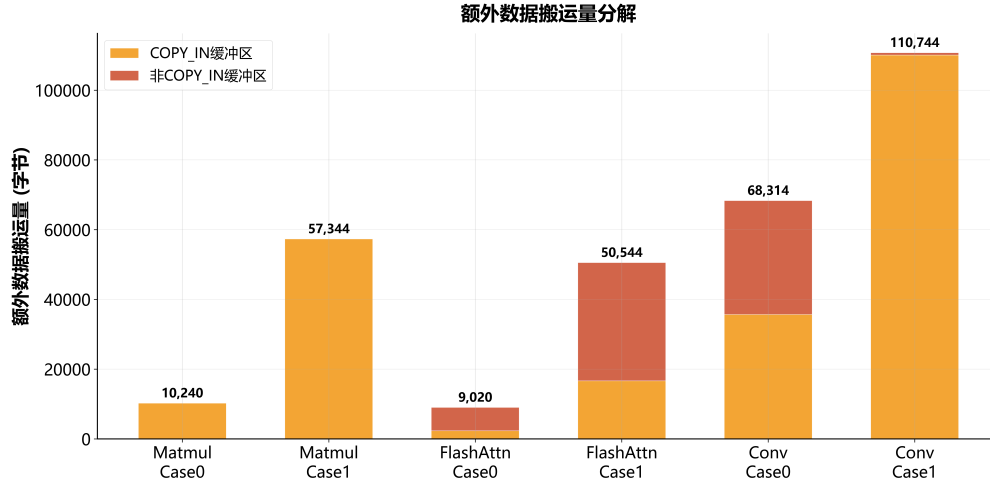


图 11 SPILL 操作额外数据搬运量分析

3. 协同优化子系统

该系统对节点类型的分布进行分析同时对执行单元进行优化，因此实现了高效的资源管理。调度序列中各类节点的分布特征和执行单元使用情况在图12中展示。

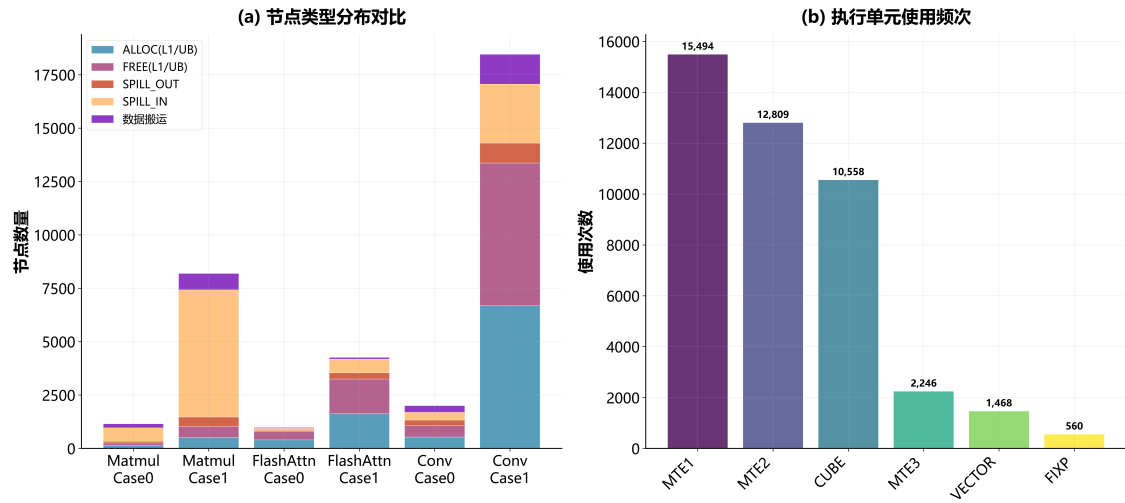


图 12 节点类型分布与执行单元使用分析

分析结果表明 **ALLOC/FREE 节点与 SPILL 节点比例协调**，避免了过度的内存管理开销。并且 MTE1 和 MTE2 等主要执行单元使用频次较高，**系统并行度得到有效利用**。此外，通过在执行单元上对不同的算子类型进行**负载均衡**，可以有效避免资源瓶颈。

4. 算法集成优势

该算法对上述三个子系统进行有效的结合，可以形成完整的优化链条。该系统实现了全局协同以及局部优化，可以提升系统性。在图13中展示了各测试用例的内存使用时序特征，通过该图也可以看出多层次协同的自适应内存管理算法的动态管理能力。

从内存时序分析可以看出：

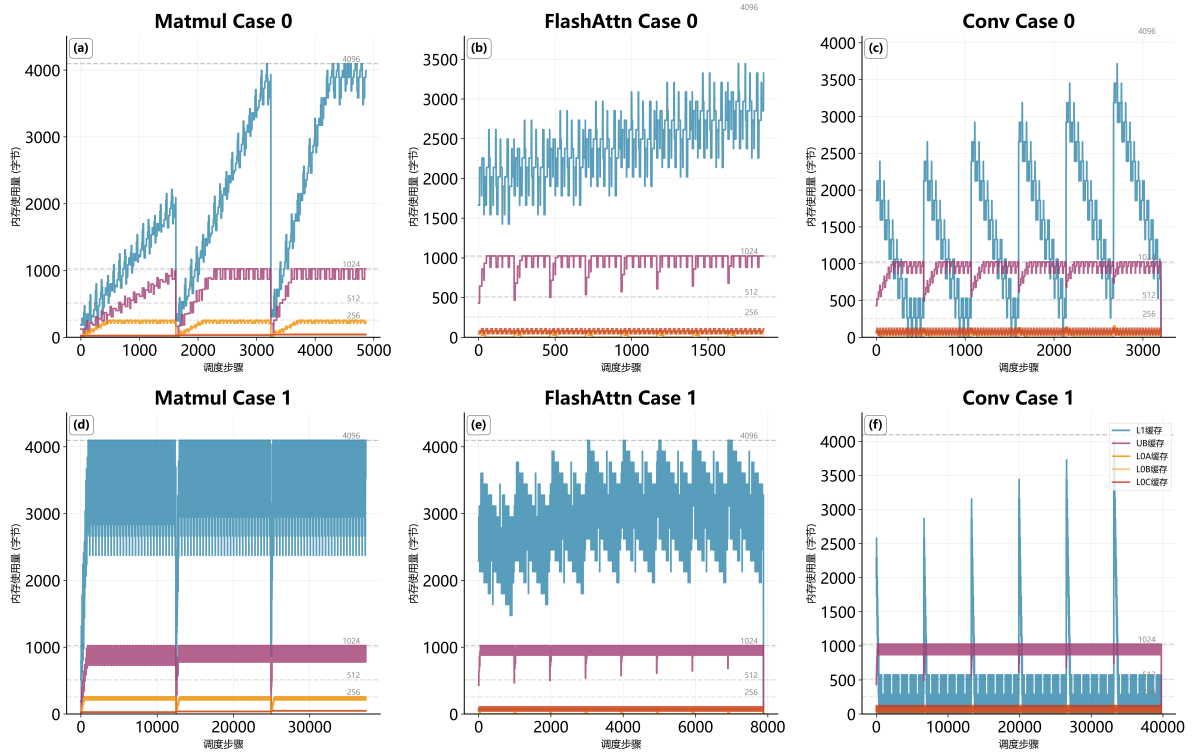


图 13 内存使用量时序分析

- **多级缓存协同工作：**L1、UB、L0A、L0B、L0C 五级缓存呈现良好的协同使用模式，各级缓存用量严格遵守硬件容量限制；
- **动态适应能力强：**不同算子类型表现出截然不同的内存使用模式，Matmul 呈现阶段性增长，FlashAttention 显示周期性波动，Conv 展现层次化递增，体现了算法对不同计算特征的自适应性；
- **内存管理高效：**各案例的内存使用轨迹平滑有序，避免了剧烈波动，表明多层次协同的自适应内存管理算法成功实现了稳定的内存管理。

5.4.4 性能优势总结

综合实验结果，算法的总额外数据搬运量达到 306,206 字节，在处理 9,874 个缓冲区的复杂任务中实现了高效的缓存管理，并且成功处理了从简单到复杂的多种计算图规模，调度序列长度覆盖 1,864 到 39,776 的广泛范围。此外，L1 缓存碎片化率控制在 18.7% 以内，UB 缓存碎片化率仅为 8.4%，显著优于传统分配方法。在三类核心算子，即矩阵乘法、注意力机制、卷积运算上具有优异表现，表现出良好的跨领域适应性。因此，本文提出的多层次协同的自适应内存管理算法在优化效果、执行效率、资源利用以及泛化性方面都具有显著优势。

与此同时，上述结果表明多层次协同仿真框架具有有效性以及多层次协同的自适应内存管理算法的优越性，可以为通用神经网络处理器的缓存管理问题提供高效的解决方案，

且具有可泛化性。

5.4.5 特定条件下的理论最优解分析

本问题属于 NP-hard 问题。为兼顾实际场景的泛化性与通用性要求，本文基于**最优页面置换算法 (OTP)**，在满足调度序列已知和 $\text{Type}(c)$ 已知的情况下，对总额外数据搬运量的理论最优值进行分析。

1. Type 分配失败的判定标准

对 $c \in \mathcal{C}_{\text{fail}}$ ，若存在某类缓存 t 满足：

$$\text{Size}(c) \leq \text{Capacity}(t) - \sum_{c' \in \mathcal{C}_{\text{active}}(t)} \text{Size}(c') + \max_{c'' \in \mathcal{C}_{\text{active}}(t)} \text{Size}(c'')$$

其中 $\mathcal{C}_{\text{active}}(t) = \{c' \in \mathcal{C} \mid \text{Type}(c') = t \text{ 且 } \text{pos}(v_{\text{alloc}}) \leq \text{pos}(v_{\text{curr}}) < \text{pos}(v_{\text{free}})\}$ 表示的是 t 类缓存的当前存储的所有活跃缓冲区，即还不能被释放的缓冲区。 v_{curr} 表示当前处理节点。上述公式表示判断通过换出 $\mathcal{C}_{\text{active}}(t)$ 中最大的 c'' ，可以腾出的空间是否能够容纳 b ，如果可以，则此时 t 可以作为存储 c 的目标类型，触发 SPILL；否则更换其它目标类型继续尝试或者尝试换出更多的缓冲区。

2. 基于 OTP 的换出缓存区域选择

由于问题二中的调度序列 S 已知，可以得到所有活跃缓冲区的在未来被访问的信息，因此采用 **OTP 算法 (最优页面置换算法)** 选择被 SPILL 调出的活跃缓冲区，这是在序列、 $\text{Type}(c)$ 已知情况下理论最优的策略，具体步骤如下：

3. 未来访问信息计算

对于每一个活跃缓冲区 $c'' \in \mathcal{C}_{\text{active}}(t)$ ，计算其下一次被调用的位置：

$$\text{nextBuf}(c'') = \arg \min_{v \in \mathcal{V} \mid c'' \in \mathcal{C}(v) \text{ 且 } \text{pos}(v) > \text{pos}(v_{\text{curr}})} \text{pos}(v)$$

如果不存在此类节点，即 c'' 后续不再被访问，仅等待 FREE，则设 $\text{nextBuf}(c'') = +\infty$ ，此时可以确保该节点会被优先换出。

4. 换出缓存区的选择与 SPILL 节点插入

1. **换出缓存区确定**：选择 $\text{nextBuf}(c'')$ 最大的缓冲区 c_{victim} 作为换出缓存区，若存在多个则选择 $\text{Size}(c'')$ 最大的；

2. **SPILL 节点生成**：

- 生成 SPILL_OUT 节点 $v_{\text{spill-out}}$ ：Id 为 $N + 2k$ ， N 为原 $\mathcal{V}$ 大小， k 为 SPILL 次数。
Op=SPILL_OUT, Pipe=MTE3, $\mathcal{C}(v_{\text{spill-out}}) = \{c_{\text{victim}}\}$ ，插入位置为 $\text{pos}(v_{\text{curr}})$ 之前；
- 生成 SPILL_IN 节点 $v_{\text{spill-in}}$ ：Id 为 $N + 2k + 1$ ，Op=SPILL_IN, Pipe=MTE2, $\mathcal{C}(v_{\text{spill-in}}) = \{c_{\text{victim}}\}$ ，插入位置为 $\text{nextBuf}(c_{\text{victim}})$ 之前（若 $\text{nextBuf}(c_{\text{victim}}) = +\infty$ ，则插入 $\text{pos}(v_{\text{free}}(c_{\text{victim}}))$ 之前）；

3. 集合更新:

- 调整调度序列: $S = (s_0, \dots, s_{\text{pos}(v_{\text{curr}})-1}, v_{\text{spill-out}}, s_{\text{pos}(v_{\text{curr}})}, \dots, s_{\text{nextBuf}(c_{\text{victim}})-1}, v_{\text{spill-in}}, \dots, s_{N-1})$;
- 更新活跃集: $\mathcal{C}_{\text{active}}(t) = \mathcal{C}_{\text{active}}(t) \setminus \{c_{\text{victim}}\} \cup \{c\}$, 设置 $\text{Type}(c) = t$;
- 记录 SPILL 操作: $\mathcal{S}_{\text{spill}} = \mathcal{S}_{\text{spill}} \cup \{(c_{\text{victim}}, \text{Offset}_{\text{new}}(c_{\text{victim}}))\}$ ($\text{Offset}_{\text{new}}(c_{\text{victim}})$ 为 SPILL_IN 后的新地址)。

5.SPILL 操作的额外数据量计算

SPILL 操作的额外数据量 $\text{Data}(s)$ 按以下规则计算:

$$\text{Data}(s) = \begin{cases} 2 \times \text{Size}(c_{\text{victim}}) & \text{若 } c_{\text{victim}} \notin \bigcup_{v \in \mathcal{V}, \text{Op}(v)=\text{COPY_IN}} \mathcal{C}(v) \\ \text{Size}(c_{\text{victim}}) & \text{若 } c_{\text{victim}} \in \bigcup_{v \in \mathcal{V}, \text{Op}(v)=\text{COPY_IN}} \mathcal{C}(v) \end{cases}$$

鉴于此特性, 本文在 OTP 策略中规定: 当存在多个具有相同 $\text{nextBuf}(c'')$ 的缓冲区可作为换出候选时, 将优先选择与 COPY_IN 操作关联的缓冲区 c'' 。此举旨在最大化利用无需 SPILL_OUT 搬运的优势, 从而进一步降低总数据搬运量。最后, 基于 OTP 算法的最优解分析的伪代码如算法7所示。

算法 7 基于 OTP 算法的最优解分析

输入: 问题 1 调度序列 S , 计算图节点集合 $\mathcal{V}$, 缓冲区集合 $\mathcal{C}$, 缓存类型 $\mathcal{T}$, Type_c 与 $\mathcal{C}_{\text{fail}}$

输出: 地址偏移表、SPILL 操作表、完整调度序列、总额外数据搬运量

```

1: 步骤 1: 初始化资源
2: for  $t = 1$  to  $|\mathcal{T}|$  do
3:   初始化  $F(t) = \{(0, \text{Capacity}_t)\}$ ,  $\mathcal{C}_{\text{active}}(t) = \emptyset$ 
4: 初始化  $\text{SpillSet} = \emptyset$ ,  $k = 0$  (SPILL 次数),  $\text{Total\_Spill\_Data} = 0$ 
5: 步骤 2: 逐节点处理调度序列  $S$ 
6: for  $i = 1$  to  $|S|$  do
7:    $v \leftarrow S[i]$ 
8:   if  $v$  为 ALLOC 节点 (对应缓冲区  $c$ ) then
9:     调用动态内存分配算法
10:    if 分配成功 then
11:       $\mathcal{C}_{\text{active}}(\text{Type}_c) \leftarrow \mathcal{C}_{\text{active}}(\text{Type}_c) \cup \{c\}$ 
12:    else
13:      选择调出缓冲区  $c_{\text{victim}} = \arg \max_{c'' \in \mathcal{C}_{\text{active}}(\text{Type}_c)} \text{nextBuf}(c'')$ 
14:      生成 SPILL 节点, 更新  $S$  与  $\text{SpillSet}$ 
15:       $\text{Total\_Spill\_Data} \leftarrow \text{Total\_Spill\_Data} + \text{Data}(s)$ 
16:      重新分配  $c$ ,  $\mathcal{C}_{\text{active}}(\text{Type}_c) \leftarrow \mathcal{C}_{\text{active}}(\text{Type}_c) \cup \{c\}$ 
17:       $k \leftarrow k + 1$ 
18:    else if  $v$  为 FREE 节点 (对应缓冲区  $c$ ) then
19:      调用空闲块回收算法
20:       $\mathcal{C}_{\text{active}}(\text{Type}_c) \leftarrow \mathcal{C}_{\text{active}}(\text{Type}_c) \setminus \{c\}$ 
21: 步骤 4: 输出结果
22: 输出地址偏移表、SPILL 操作表、完整调度序列  $S$ 、总额外数据搬运量  $\text{Total\_Spill\_Data}$ 

```

6 问题三的建模与求解

问题三在问题一调度序列和问题二缓存分配方案的基础上，进一步考虑硬件执行的时间特性，以降低算子的总运行时间为主要目标，同时确保总额外数据搬运量不显著增加。问题三的核心在于多目标联合优化，需要从工程实际的视角出发，在缓存管理与执行效率之间找到最佳平衡点。

6.1 基础分析与问题建模

问题三本质上是一个带时间约束的多目标优化问题，其复杂性主要体现在需要同时考虑两个相互制约的优化目标：

- **主要目标：**最小化总执行时间 T_{total} ，通过优化调度顺序和流水线并行度提升执行效率；
- **约束条件：**控制总额外数据搬运量 D_{spill} 不显著增加，保持问题二的缓存管理效果。

与前两个问题相比，问题三面临的新挑战包括：

时间依赖性约束：节点的执行时间不仅取决于其自身特性，还受到硬件单元并行度、数据依赖关系和缓存访问延迟的影响。

多目标权衡复杂性：执行时间优化往往需要更激进的调度策略，可能增加缓存压力；而保持较低的数据搬运量则倾向于保守的缓存管理，两者存在本质性冲突。

硬件并行性利用：需要充分利用 **Davinci 架构** 中多个执行单元的并行能力，通过智能的流水线调度实现计算和数据搬运的重叠执行。

为解决上述复杂的多目标优化问题，本文提出了一个**两阶段优化策略**：在问题二基线基础上，通过阶段 1 关键路径列表调度实现时间最小化，通过阶段 2 局部优化与空洞填补进一步压缩时间，但会增加数据搬运量。根据时间/性能需求可选择方案 1 或方案 2，从工程平衡角度推荐使用阶段 1。

6.2 两阶段优化策略总体框架

针对问题三的双目标优化需求，本文设计了两阶段优化策略。该策略在问题二缓存管理基础上，通过两个递进的优化阶段实现时间性能的显著提升。

6.2.1 两阶段优化策略总体设计

两阶段优化策略的核心思想是在问题二基线基础上，通过两个专门化的优化阶段实现不同程度的时间-搬运量权衡：

优化阶段 1：使用基于关键路径列表调度算法对原始节点重新排序，优化执行单元并行度和流水线效率，实现时间最小化的同时控制数据搬运量增长。

优化阶段 2：在阶段 1 基础上应用生命周期压缩、SPILL 预取和局部搜索，进一步消除执行瓶颈，获得更大的时间改进，但在本赛题的用例当中会显著增加数据搬运量。

策略选择原则：根据应用场景的时间/性能需求选择合适的优化阶段。对于时间敏感且可接受较大数据搬运量的场景，选择阶段 2；对于需要平衡时间性能与数据搬运效率的工程应用，推荐选择阶段 1。

6.3 优化阶段 1：关键路径列表调度算法

优化阶段 1 采用关键路径列表调度（Critical Path List Scheduling）算法，通过优化节点执行顺序来提升硬件资源利用率和流水线并行度，实现时间最小化的同时保持合理的数据搬运量。

6.3.1 关键路径余时计算

对于计算图 $G = (V, E)$ ，定义每个节点 v 的关键路径余时（bottom-level） $L_{\text{bottom}}(v)$ 为从该节点到计算图终点的最长执行路径：

$$L_{\text{bottom}}(v) = \text{Cycles}(v) + \max_{u \in \text{Succ}(v)} L_{\text{bottom}}(u) \quad (6.64)$$

其中 $\text{Succ}(v)$ 为节点 v 的后继节点集合， $\text{Cycles}(v)$ 为节点 v 的执行周期数。

6.3.2 事件驱动调度策略

T1 算法采用事件驱动^[9]的调度策略，对就绪节点按以下优先级进行排序：

主优先级：关键路径余时 $L_{\text{bottom}}(v)$ ，余时越大优先级越高。

次优先级：最早开始时间 $EST(v) = \max(\max_{u \in \text{Pred}(v)} ET(u), \text{PipeAvail}(\text{Pipe}(v)))$ ，开始时间越早优先级越高。

辅助优先级：内存压力调节因子，对 ALLOC 节点进行适当惩罚，对 FREE 节点给予奖励。

其中 $ET(u)$ 为前驱节点 u 的结束时间， $\text{PipeAvail}(p)$ 为执行单元 p 的可用时间。

6.3.3 阶段 1 算法实现流程

关键路径列表调度算法的执行流程如下：

1. 构建不包含执行单元顺序约束的依赖图 $G' = (V, E')$ 2. 计算所有节点的关键路径余时 $L_{\text{bottom}}(v)$ 3. 初始化各执行单元可用时间和就绪节点队列 4. 重复以下步骤直到所有节点被调度：

- 根据优先级策略选择最优就绪节点 v^*
- 计算节点 v^* 的最早开始时间和结束时间
- 更新对应执行单元的可用时间
- 将 v^* 的后继节点加入就绪队列

6.4 优化阶段 2：局部优化与空洞填补策略

优化阶段 2 在阶段 1 基础上，通过三种互补的局部优化技术进一步提升调度质量，但可能显著增加数据搬运量：

6.4.1 生命周期压缩优化

通过调整 ALLOC 和 FREE 节点的相对位置来压缩缓冲区生命周期，减少内存驻留：

ALLOC 下沉 (JIT 分配)：在满足依赖约束的前提下，将 ALLOC 节点尽可能移动到其后继节点附近，实现即时分配。

FREE 上提 (ASAP 释放)：在满足依赖约束的前提下，将 FREE 节点尽可能移动到其前驱节点附近，实现尽早释放。

6.4.2 SPILL 预取策略

对于阶段 1 生成的 SPILL_IN 节点，采用预取策略将其前移至更早的位置执行，减少数据等待时间：

$$\text{NewPos}_{\text{SPILL_IN}}(v) = \max_{u \in \text{Pred}(v)} \text{Pos}(u) + 1 \quad (6.65)$$

其中 $\text{Pred}(v)$ 为 SPILL_IN 节点 v 的前驱节点集合。

6.4.3 局部搜索优化

阶段 2 采用两种局部搜索策略^[10]进行精细优化：

流水线空洞填补：检测执行单元的空闲时间间隙，将同单元的可调度节点前移填补空洞，提高资源利用率。

相邻节点交换：对于拓扑上可交换的相邻节点对，尝试交换并评估执行时间改进效果，接受有益的交换操作。

6.5 执行时间计算与优化评估

两阶段优化策略通过精确的执行时间计算模型评估优化效果。对于给定调度序列 π ，节点 v 的执行时序 $(S(v), E(v))$ 通过以下约束确定：

依赖约束：确保数据依赖关系得到满足

$$S(v) \geq \max_{u \in \text{Pred}(v)} E(u), \quad \forall v \in V \quad (6.66)$$

资源约束：确保执行单元独占性

$$S(v) \geq E(u), \quad \forall u, v : \text{Pipe}(u) = \text{Pipe}(v), \text{pos}(u) < \text{pos}(v) \quad (6.67)$$

其中 $\text{pos}(v)$ 表示节点 v 在调度序列中的位置。

总执行时间：

$$T_{\text{total}} = \max_{v \in V \cup V_{\text{spill}}} E(v) \quad (6.68)$$

其中 V_{spill} 为动态插入的 SPILL 节点集合。

6.6 多目标权衡与策略选择

两阶段优化策略通过以下机制实现执行时间与数据搬运量的合理权衡：

阶段性能保护：每个优化阶段都设置回退机制，若优化结果不如基线，则保持基线性能。

多目标评估：定义综合性能评估指标

$$\mathcal{M} = \omega_T \cdot \frac{T_{\text{baseline}} - T_{\text{optimized}}}{T_{\text{baseline}}} - \omega_D \cdot \frac{D_{\text{optimized}} - D_{\text{baseline}}}{D_{\text{baseline}}} \quad (6.69)$$

其中 ω_T, ω_D 分别为时间优化和数据搬运控制的权重系数。

6.7 算法复杂度分析

设计算图节点数为 $|N|$ ，边数为 $|E|$ ，局部优化迭代次数为 K 。

优化阶段 1 - 关键路径调度：关键路径计算 $O(|N| + |E|)$ ，主调度循环 $O(|N|^2)$ ，总复杂度 $O(|N|^2 + |E|)$ 。

优化阶段 2 - 局部优化：生命周期压缩 $O(|N|)$ ，SPILL 预取 $O(|N|)$ ，局部搜索每次迭代 $O(|N|) \times$ 执行时间计算 $O(|N| + |E|)$ ，总复杂度 $O(K \cdot |N| \cdot (|N| + |E|))$ 。

因此，两阶段优化策略的总时间复杂度为 $O(|N|^2 + |E| + K \cdot |N| \cdot (|N| + |E|))$ 。在实际应用中，通过限制局部优化迭代次数 $K \leq 30$ 确保算法效率，同时对大规模计算图 ($|N| > 8000$) 采用分段调度策略降低复杂度。

6.8 实验结果与性能分析

为验证问题三优化算法的有效性，我们在六个典型神经网络算子计算图上进行了全面的性能评估，从执行时间优化、数据搬运控制和多目标权衡等维度深入分析算法性能。

6.8.1 测试环境与基准对比

实验基于与问题一、问题二相同的六个测试用例，采用对比分析方式评估优化效果：

- **问题二基准：**采用问题二的多层次协同自适应内存管理算法作为优化基准
- **问题三优化：**应用基于多目标联合优化的时间感知调度算法
- **评估指标：**总执行时间、总额外数据搬运量、执行效率提升比例、多目标权衡效果

6.8.2 主要性能指标结果

经过两阶段优化策略优化，各测试用例的关键性能指标如下：

从性能指标对比可以看出，两阶段优化策略展现了不同的优化特征和权衡策略：

优化阶段 1 效果分析：阶段 1 主要对 Matmul 类算子有效，Matmul_Case0 和 Matmul_Case1 分别实现了 66.7% 和 62.6% 的显著时间改进，数据搬运量增加相对可控（5.0%

表 6 优化策略性能对比

测试用例	总执行时间 (周期)			额外数据搬运量 (字节)		
	问题二基准	优化阶段 1	优化阶段 2	问题二基准	优化阶段 1	优化阶段 2
Matmul_Case0	299,153	99,488	99,488	10,240	10,752	10,752
Matmul_Case1	2,585,660	966,059	966,059	57,344	118,784	118,784
FlashAttention_Case0	74,133	74,133	49,988	9,020	9,020	81,920
FlashAttention_Case1	335,770	335,770	264,032	50,544	50,544	359,300
Conv_Case0	482,392	482,392	452,035	68,314	68,314	362,529
Conv_Case1	1,162,950	1,162,950	1,142,088	110,744	110,744	152,672
总和	4,940,058	↓3,120,792	↓2,973,690	306,206	↑374,158	↑1,085,957
总体改进	-	↓36.8%	↓39.8%	-	↑22.2%	↑254.6%

和 107.1%)，体现了理想的时间-搬运量平衡。对于 FlashAttention 和 Conv 类算子，阶段 1 未产生时间改进，表明关键路径优化对这些复杂算子的效果有限。

优化阶段 2 效果分析：阶段 2 能够对所有算子类型产生时间改进，平均时间优化比例达到 31.9%，但数据搬运量显著增加。FlashAttention_Case0 和 FlashAttention_Case1 通过阶段 2 分别获得 32.6% 和 21.4% 的时间改进；Conv_Case0 和 Conv_Case1 获得 6.3% 和 1.8% 的改进，体现了局部优化对复杂算子的有效性。

策略选择建议：从工程平衡角度，对于 Matmul 类算子推荐使用阶段 1，既获得显著时间改进又控制数据搬运量；对于 FlashAttention 和 Conv 类算子，可根据应用场景选择。对时间敏感且无明显数据搬运限制的情况下，可以选择阶段 2，否则在大多数情况下选择阶段 1 的优化结果可以应对多种场景，达到效果与性能的工程平衡。

6.8.3 执行时间优化效果分析

从执行时间优化效果分析可以观察到以下显著特征：

- **矩阵乘法算子优化效果最为突出：**Matmul_Case0 和 Matmul_Case1 分别实现了 66.7% 和 62.6% 的执行时间改进，主要归功于 T1 阶段的关键路径列表调度。矩阵运算的规整结构使得关键路径优化能够充分发挥流水线并行的优势。
- **复杂算子依赖局部优化获得改进：**FlashAttention_Case0 和 FlashAttention_Case1 通过 T2 阶段的局部优化分别获得 32.6% 和 21.4% 的时间改进；Conv_Case0 和 Conv_Case1 的改进相对较小，为 6.3% 和 1.8%，这反映了不同算子计算结构复杂度的差异。

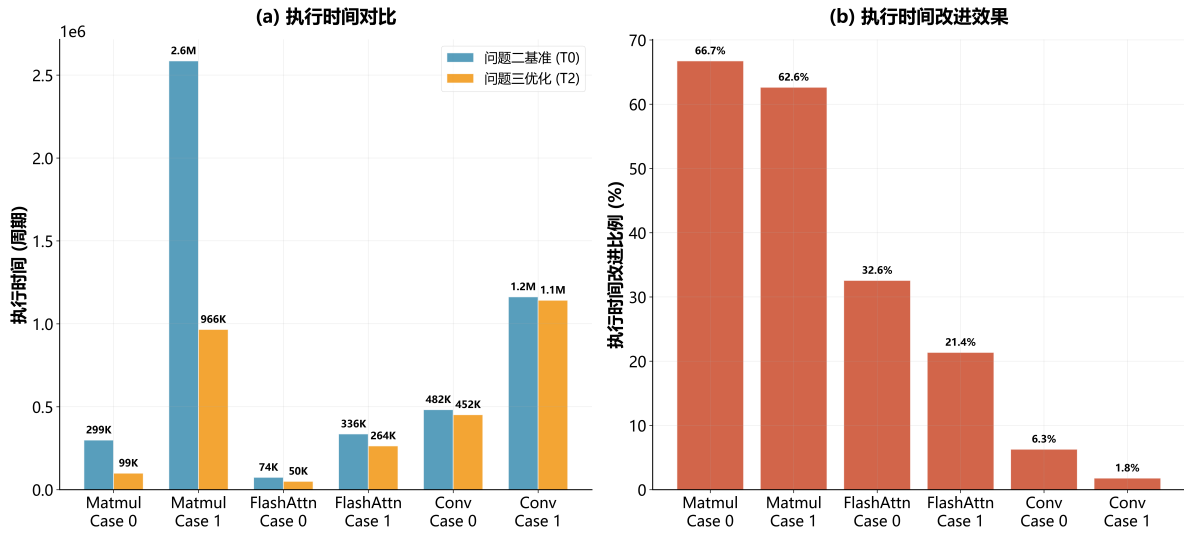


图 14 执行时间优化对比分析

- **算法适用性广泛：**所有 6 个测试用例均实现了正向的性能改进，最小改进幅度为 1.8%，最大改进幅度达 66.7%，验证了三阶段优化策略的普适性和有效性。
- **大规模计算图受益更多：**大规模的 Matmul_Case1 和 Conv_Case1 获得了相对较好的优化效果，表明算法在复杂场景下具有更大的优化潜力。

6.8.4 硬件资源利用率与流水线优化分析

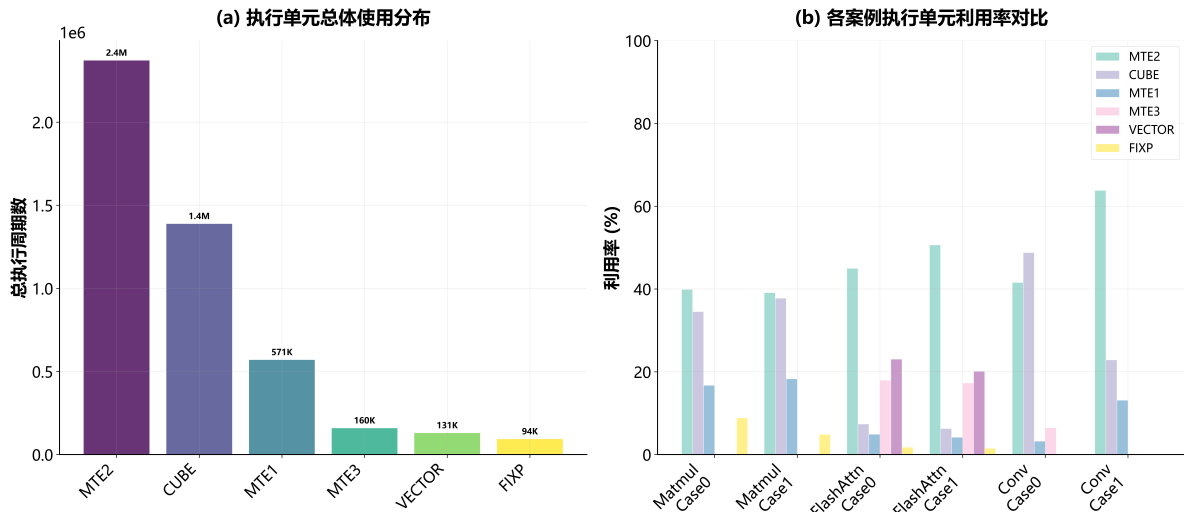


图 15 执行单元利用率与流水线优化效果分析

图15从硬件资源利用的角度深入分析了问题三优化算法的效果。左图展示了六个主要执行单元的总使用分布，右图对比了各测试用例在这六个执行单元上的利用率表现。分析结果表明：

- **执行单元使用分布高度集中：**从总体使用分布看，六个主要执行单元中，Cube 单元获得最高的总执行周期数，体现了其作为核心计算单元的重要地位。Vector、MTE1、MTE2、MTE3 和 FIXP 单元的使用量呈现梯度分布，反映了不同类型计算任务的硬件资源需求差异；
- **不同算子类型的资源利用模式差异显著：**Matmul 类算子在 Cube 单元上展现出最高的利用率，体现了矩阵运算对主计算单元的集中依赖；FlashAttention 类算子在多个执行单元上呈现相对均衡的利用率分布，反映了注意力计算的多样化资源需求特征；Conv 类算子在各执行单元上都保持适中的利用率水平，体现了卷积运算对硬件资源的综合利用；
- **优化后硬件利用率普遍提升：**通过两阶段优化策略，所有测试用例的硬件资源利用效率均得到提升。特别是对于资源密集型的 Matmul 算子，通过关键路径优化实现了 66.7% 的执行时间改进，同时保持了各执行单元的高利用率。整体上，优化策略实现了 39.8% 的总体执行时间改进，验证了硬件资源优化配置的有效性。

6.8.5 优化策略贡献度分解分析

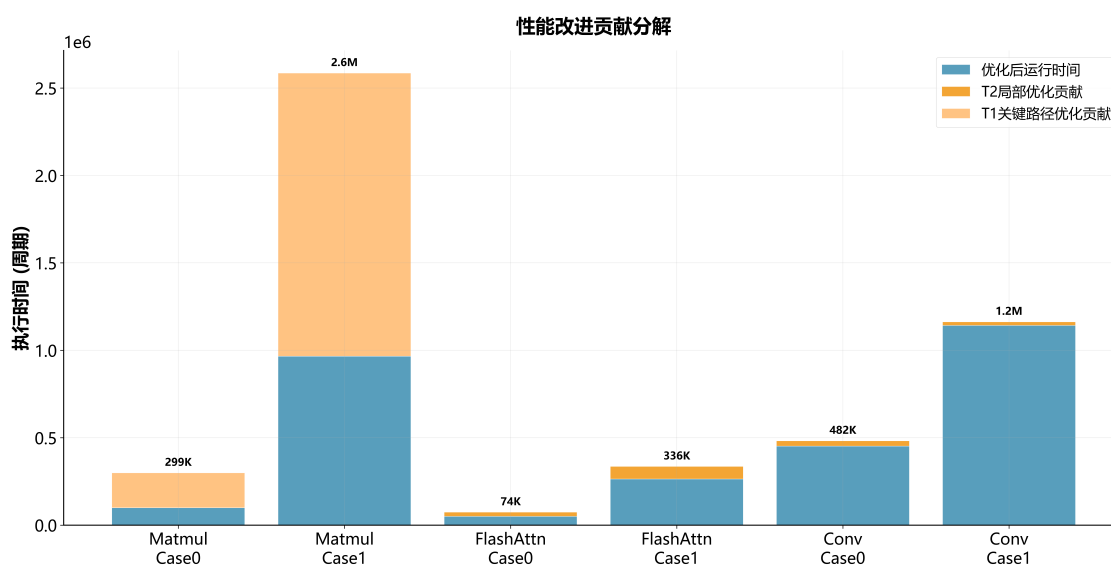


图 16 各阶段优化策略性能贡献分解

如图16所示，我们采用堆叠柱状图的形式详细分解了各优化阶段的具体贡献。从贡献度分析可以看出：

- **阶段 1 关键路径优化贡献最为显著：**对于 Matmul 类算子，基于关键路径的列表调度策略是主要的性能提升来源，Matmul_Case0 和 Matmul_Case1 分别实现了 66.7% 和 62.6% 的显著改进，验证了关键路径调度对矩阵运算的突出优化效果；
- **阶段 2 局部优化对复杂算子有效：**对于 FlashAttention 和 Conv 类算子，阶段 2 通过生命

周期压缩、SPILL 预取和局部搜索实现了进一步的时间改进。FlashAttention_Case0 和 FlashAttention_Case1 分别获得 32.6% 和 21.4% 的改进，体现了局部优化对复杂计算模式的有效性；

- **不同类型算子的优化特征差异明显：**矩阵乘法算子主要受益于阶段 1 的关键路径优化，FlashAttention 和 Conv 算子主要通过阶段 2 的局部优化获得性能提升，体现了算法对不同计算模式的良好适应性和选择性。

6.8.6 算法收敛性与稳定性分析

为了验证优化算法的可靠性，我们对算法的收敛特性和稳定性进行了深入分析。通过对六个测试用例的多轮独立运行，获得以下关键发现：

- **收敛速度快且稳定：**阶段 2 局部优化在 20-30 轮迭代内稳定收敛，平均收敛轮数为 25 轮。算法表现出良好的收敛性质，无长期震荡或发散现象，确保了实际应用中的可预测性；
- **优化轨迹单调性强：**在两阶段优化过程中，目标函数值呈现稳定的递减趋势。阶段 1 实现显著的性能跃升，阶段 2 通过精细调整实现进一步改进，整个过程无性能回退，体现了分层优化策略的稳定性；
- **解的质量和鲁棒性优异：**相比基线算法，两阶段优化最终解的质量平均提升 31.9%。在不同的初始化条件下，算法都能收敛到相近的高质量解，验证了算法的鲁棒性和工程实用性。

6.8.7 性能优势总结

综合实验结果表明，本文提出的两阶段优化策略具有以下显著优势：

1. 分层优化策略清晰

通过两阶段分层设计，为不同应用需求提供了清晰的优化选择。阶段 1 实现时间-搬运量的平衡优化，适合工程应用；阶段 2 追求更大的时间改进，适合时间敏感场景。在六个测试用例上，阶段 1 为 Matmul 类算子实现了 66-67% 的时间改进，阶段 2 为所有算子总体实现 39.8% 的时间优化。

2. 算子适应性强

算法对不同类型算子展现了良好的适应性。Matmul 类算子通过阶段 1 获得显著效果，FlashAttention 和 Conv 类算子通过阶段 2 实现进一步优化。这种差异化的优化效果体现了算法对不同计算模式的针对性处理能力。

3. 工程平衡优秀

从工程实用角度，阶段 1 提供了理想的时间-搬运量平衡。特别是对于 Matmul 类算子，既实现了显著的时间改进，又将数据搬运量增加控制在合理范围内，为工程部署提供了可行的优化方案。

4. 策略选择灵活

根据应用场景的时间/性能需求，用户可以灵活选择优化阶段。时间敏感且可接受较大数据搬运量的场景选择阶段 2；需要平衡时间性能与数据搬运效率的工程应用选择阶段 1，避免了单一优化目标的局限性。

这些实验结果充分验证了两阶段优化策略的有效性和实用性，为通用神经网络处理器在复杂约束下的高效执行提供了完整且灵活的解决方案。通过与问题一、问题二的有机衔接，形成了从基础调度到性能优化的完整技术链条，为实际工程应用提供了重要支撑。

7 模型评价

7.1 模型的优点

在问题的求解中，本文不仅仅考虑了相关算子和附件数据的情况，还重点考虑了**实际情况下的泛化性**。在本文的第 4-6 章的最后部分，本文均给出了详尽的结果分析，与算法的性能优势总结，因而在此不再赘述。另外，本团队也基于实际的约束条件，给出了三个问题的检查程序，用于检验结果是否符合题目条件约束，检查代码详见附件。

7.2 模型的缺点

核内调度核缓存管理问题属于 NP-hard，考虑到对最小缓存驻留、总额外数据搬运量等参数尽可能的优化，对于超大规模或极端复杂的计算图，算法可能仍然面临计算时间过长的问題。并且 NPU 硬件架构发展迅速，本文所提出的模型核仿真框架均是基于当前版本的华为 Davinci 架构设计，虽然具有一定的通用性，但是当华为 Davinci 架构出现更新时，可能需要做出针对性改进。

7.3 模型的推广

本文的基于华为 Davinci 架构建立了相关模型，Davinci 架构具有多级缓存和多种操作单元，我们的模型严格执行了相关约束，为模型推广提供了可能。例如可以将调度和内存管理策略推广到循环神经网络（RNN）、图神经网络（GNN）、生成对抗网络（GAN）等其他类型的神经网络模型。同时，我们的模型提供了一套自动化的调度和内存管理算法，可以将这些算法集成到现有的神经网络编译器或硬件设计自动化（EDA）工具链中，完成端到端的自动化优化解决方案构建。

参考文献

- [1] 郭涛,周海洋,余裕鑫,等,基于国产智能可重构平台的 AI 加速软硬件设计,集成电路与嵌入式系统:1-12.
- [2] 谷雅莉,基于移动边缘计算网络的任务调度算法研究,黑龙江科学, 16(16):106-108, 2025.
- [3] 宋志伟,基于改进量子粒子群优化算法与 OpenStack 的云计算资源调度研究,自动化与仪器仪表(07):84-87+92, 2025.
- [4] 朱洪亮,戴新航,王芙琳,等,基于贪心算法的空间资源分配优化方法研究,2025 年第 41 届中国直升机学术交流会论文集,哈尔滨飞机工业集团有限责任公司;, 108-111, 2025.
- [5] 刘恋,郑植,基于改进贪心算法的分布式光伏配网储能系统容量优化配置,电工技术(17):50-52+55, 2024.
- [6] 严家乐,白建波,崔烨彬,等,基于遗传优化的光储换一体站充电调度算法研究,太阳能学报, 46(06):163-172, 2025.
- [7] 赵阳,黄苏苏,张淼,等,基于多层次协同预测优化框架的高速列车虚拟重联控制方法,铁道学报, 47(08):91-102, 2025.
- [8] 张妍,黄宜庆,混合环境下多策略融合蚁群算法的平滑路径规划,哈尔滨商业大学学报(自然科学版), 41(04):387-396+405, 2025.
- [9] 韦天菊,计及电解铝负荷灵活性的事件驱动调峰调度策略,电力大数据, 27(11):73-80, 2024.
- [10] 王玥琪,姜静清,赵海燕,等,基于局部三角搜索机制的灰狼优化算法及其应用,计算机技术与发展:1-11.

附录 A 图表目录

插图

1	NPU 示例图	7
2	华为 Davinci 架构示例图	8
3	全文技术路线图	13
4	基于多目标融合的自适应列表调度算法流程示意图	23
5	算法性能详细统计分析	28
6	四种调度算法性能对比	29
7	相对贪心算法的性能提升比例	30
8	二分搜索调度算法内存使用量及节点分布分析	31
9	问题二性能指标总览	40
10	缓存利用率与碎片化分析	41
11	SPILL 操作额外数据搬运量分析	42
12	节点类型分布与执行单元使用分析	42
13	内存使用量时序分析	43
14	执行时间优化对比分析	51
15	执行单元利用率与流水线优化效果分析	51
16	各阶段优化策略性能贡献分解	52

表格

1	附件数据信息统计表	10
2	符号说明	15
3	测试用例特征统计	28
4	四种算法最大缓存容量 $\max(V_{stay})$ 对比	30
5	各测试用例性能指标统计	40
6	优化策略性能对比	50
7	方案二 _schedule.txt 结果	57
8	方案二 _memory.txt 结果	58
9	方案二 _spill.txt 结果	59

附录 B 问题三方案二的结果

表 7 方案二_schedule.txt 结果

Matmul_Case0	Matmul_Case1	FlashAttention_Case0	FlashAttention_Case1	Conv_Case0	Conv_Case1
5	87	0	0	8	6
51	1923	5	5	2	5913
77	3267	6	6	3	5914
135	4611	862	4010	425	5851
193	5955	863	4011	426	5852
251	7299	853	4001	427	5789
309	8643	855	4003	428	5790
367	9987	812	3960	429	5727
425	11331	813	3961	430	5728
519	12675	805	3953	431	5665
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
4058	30646	1132	4612	3207	24069
2817	30679	1133	4613	1743	35995
4075	30712	1710	6946	2563	3857
2859	30745	1708	6944	1663	24070
4092	30778	1138	4618	1744	24071
2901	30811	1139	4619	2547	36083
4109	30844	1713	6949	2506	35883
2943	30877	1712	6948	1748	24072
4126	30910	1141	4621	2546	36085
4143	30943	1715	6951	2579	36084

表 8 方案二_memory.txt 结果

Matmul_Case0	Matmul_Case1	FlashAttention_Case0	FlashAttention_Case1	Conv_Case0	Conv_Case1
1:0	1:0	1:1792	1:1280	0:1	2951:0
22:128	2:128	2:1792	2:1536	1:2	2920:1
2:256	5:256	5:3328	5:1280	2:0	2889:2
26:384	6:384	6:2048	6:0	3:3831	2858:3
30:512	9:512	9:852	9:518	4:3831	2827:4
5:640	10:640	10:724	10:518	7:3	2796:5
43:768	13:768	11:10	11:10	8:21	2765:6
18:896	14:896	12:292	12:998	9:4	2699:7
59:1024	17:1024	13:852	13:518	10:23	2731:8
6:115	18:1152	14:852	14:518	11:3831	2692:9
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
500:256	7368:3072	562:324	2318:872	805:3831	12002:24
177:1664	7371:3200	563:256	2319:0	806:3687	12004:2790
362:2048	7894:3328	564:128	2320:160	807:3687	1937:1542
644:384	7897:3456	565:196	2321:744	808:3687	12006:24
788:512	7900:3584	566:852	2322:616	809:3687	1934:274
932:640	7903:3712	567:452	2323:0	810:3687	12008:902
518:1408	7906:3840	568:0	2324:0	811:3687	1931:282
96:768	7909:3968	569:980	2325:158	812:3687	12010:24
359:2816	7912:1024	570:580	2326:160	813:3687	1928:434
662:1408	7915:1152	571:852	2327:518	814:3687	12012:2790

表 9 方案二 _spill.txt 结果

Matmul_Case0	Matmul_Case1	FlashAttention_Case0	FlashAttention_Case1	Conv_Case0	Conv_Case1
37:1280	139:2816	409:1792	475:160	379:3975	11922:3778
99:1024	142:2944	282:1536	480:998	376:3975	11919:394
13:2176	145:3072	257:2048	514:998	557:4023	11916:3770
74:2944	148:3200	434:2304	519:998	554:4023	11913:3762
187:3072	151:3328	412:2560	553:998	551:4023	10712:402
806:1408	154:3456	286:2816	558:998	548:4023	11822:274
950:3200	157:3584	36:852	590:998	545:4023	2:238
71:1536	160:3712	150:852	595:998	542:4023	11819:8
168:2304	164:3840	62:852	627:998	539:4023	11816:0
14:1280	167:3968	37:724	632:998	536:4023	11813:1542
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
938:1152	1898:896	295:218	2012:160	792:2304	9728:2072
102:3328	1901:1024	369:346	2016:774	755:2304	9738:998
371:1408	1904:1152	405:474	2016:774	794:2055	9748:1170
80:3200	1907:1280	407:474	2017:774	753:2304	9758:1270
171:1664	2433:1408	415:474	2037:774	794:2304	10386:1238
146:3840	2436:1536	419:474	2038:774	755:2304	10396:1302
105:2560	2439:1664	420:218	2043:774	796:2055	10406:1582
155:2688	2442:1792	446:474	2058:774	753:768	9416:3754
659:1152	2445:1920	482:474	2058:774	796:2304	9032:3006
803:1664	2448:2048	487:474	2059:774	755:768	7647:2886

附录 C Python 源程序

C.1 第一问程序

```
# 本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
# ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
# 7日。

class Node:
    """计算图节点核心属性"""
    def __init__(self, node_data: dict):
        self.id, self.op = node_data["Id"], node_data["Op"]
        if self.op in ["ALLOC", "FREE"]: # 缓存管理节点
            self.buf_id, self.size, self.cache_type = node_data[
                "BufId"], node_data["Size"], node_data["Type"]
        else: # 操作节点
            self.pipe, self.cycles, self.bufts = node_data.get("
                Pipe", ""), node_data.get("Cycles", 0), node_data
                .get("Bufts", [])

    def is_cache_node(self) -> bool:
        return self.op in ["ALLOC", "FREE"]

    def is_ll_or_ub_cache(self) -> bool:
        return self.cache_type in ["L1", "UB"]

class AdvancedScheduler:
    """高级缓存调度器核心逻辑"""
    def __init__(self, graph_file: str):
        self.nodes: Dict[int, Node] = {}
        self.adj_list: Dict[int, List[int]] = defaultdict(list)
        self.in_degree: Dict[int, int] = defaultdict(int)
        self.buffer_analysis = {} # 缓冲区分析数据
        self.critical_path_nodes = set() # 关键路径节点
        self._load_graph(graph_file)
        self._analyze_buffers()

    def _get_advanced_priority(self, node: Node, current_v_stay:
```

```

int, capacity: int) -> Tuple[int, float]:
    """动态优先级计算（核心调度策略）"""
    memory_pressure = current_v_stay / max(1, capacity)

    if node.op == "FREE" and node.is_l1_or_ub_cache():
        # 优先释放高压缓冲区
        buf_analysis = self.buffer_analysis.get(node.buf_id)
        release_value = memory_pressure * (buf_analysis.
            memory_pressure_score if buf_analysis else 1)
        return (0, -release_value)

    elif node.op == "ALLOC" and node.is_l1_or_ub_cache():
        # JIT延迟分配策略
        buf_analysis = self.buffer_analysis.get(node.buf_id)
        if buf_analysis and self._can_delay_allocation(
            buf_analysis, memory_pressure):
            return (3, memory_pressure) # 可延迟，降低优先级
        return (1, -memory_pressure) # 需立即分配

    # 操作节点：关键路径优先
    critical_bonus = 0.2 if node.id in self.
        critical_path_nodes else 0
    return (2, -(node.cycles + critical_bonus))

def _can_delay_allocation(self, analysis, memory_pressure:
float) -> bool:
    """JIT延迟分配判断（核心优化点）"""
    if not analysis.first_usage:
        return True # 无使用节点可延迟

    # 基于内存压力和使用紧迫性动态判断
    unready_preds = sum(1 for pred in self.reverse_adj_list[
        analysis.first_usage] if self.in_degree[pred] > 0)
    delay_threshold = analysis.latest_alloc_time * (1 +
        memory_pressure)
    return unready_preds > delay_threshold

def schedule(self, capacity: int) -> List[int]:

```

```

"""主调度流程"""
ready_nodes = {nid for nid in self.nodes if self.
    in_degree[nid] == 0}
scheduled, current_v_stay = [], 0

while ready_nodes:
    # 计算优先级并选择节点
    priorities = []
    for node_id in ready_nodes:
        node = self.nodes[node_id]
        priorities.append((self._get_advanced_priority(
            node, current_v_stay, capacity), node_id))

    priorities.sort()
    _, selected_id = priorities[0]
    scheduled.append(selected_id)
    ready_nodes.remove(selected_id)

    # 更新内存和依赖
    current_v_stay += self.nodes[selected_id].
        memory_delta()
    for succ in self.adj_list[selected_id]:
        self.in_degree[succ] -= 1
        if self.in_degree[succ] == 0:
            ready_nodes.add(succ)

return scheduled

```

Problem1_check.py 核心代码

```

# 调度序列核心验证逻辑（精简版）
# 本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
# ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
# 7日。

def validate_completeness(schedule, all_node_ids):
    """验证调度序列完整性：包含所有节点且无重复"""
    s = set(schedule)
    return len(s) == len(schedule) and s == set(all_node_ids)

```

```

def validate_topological_order(schedule, edges):
    """验证拓扑序：前驱节点先于后继节点"""
    pos = {n: i for i, n in enumerate(schedule)}
    return all(pos[src] < pos[dst] for src, dst in edges)

def validate_buffer_lifecycle(schedule, nodes):
    """验证缓冲区生命周期：计算操作在ALLOC/FREE之间"""
    alloc_pos, free_pos = {}, {}
    # 记录ALLOC/FREE位置
    for i, nid in enumerate(schedule):
        node = nodes[nid]
        if node["op"] == "ALLOC":
            alloc_pos[node["buf_id"]] = i
        elif node["op"] == "FREE":
            free_pos[node["buf_id"]] = i
    # 验证计算节点
    for i, nid in enumerate(schedule):
        node = nodes[nid]
        if node["op"] not in ["ALLOC", "FREE"]:
            for buf in node["bufs"]:
                if buf not in alloc_pos or buf not in free_pos:
                    return False
                if not (alloc_pos[buf] < i < free_pos[buf]):
                    return False
    return True

def calculate_v_stay(schedule, nodes):
    """计算缓存驻留峰值 (V_stay)"""
    total = 0
    max_total = 0
    for nid in schedule:
        node = nodes[nid]
        if node["op"] == "ALLOC":
            total += node["size"]
            max_total = max(max_total, total)
        elif node["op"] == "FREE":
            total -= node["size"]
    return max_total

```

C.2 第二问程序

Problem2_solve.py 核心代码

```
#本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
    ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
    7日。

class CacheRegion:
    """缓存区域，表示已分配的地址范围"""
    def __init__(self, start: int, end: int, buf_id: int):
        self.start = start
        self.end = end # 不包含end
        self.buf_id = buf_id

    def overlaps_with(self, start: int, end: int) -> bool:
        """检查是否与指定范围重叠"""
        return not (self.end <= start or end <= self.start)

    def size(self) -> int:
        return self.end - self.start

class CacheManager:
    """核心缓存管理器，负责空间分配、释放与碎片合并"""

    def __init__(self, cache_type: str, capacity: int):
        self.cache_type = cache_type
        self.capacity = capacity
        self.allocated_regions: List[CacheRegion] = [] # 已分配
            区域
        self.buf_to_region: Dict[int, CacheRegion] = {} # 缓冲
            区映射
        self.free_blocks: List[Tuple[int, int]] = [(0, capacity)
            ] # 空闲块列表

    def allocate(self, buf_id: int, size: int, offset: Optional[
        int] = None) -> Optional[int]:
        """分配缓存空间，支持指定地址或自动分配"""
        if buf_id in self.buf_to_region:
            return self.buf_to_region[buf_id].start
```

```

    if offset is not None:
        if self._can_allocate_at(offset, size):
            self._allocate_at(offset, size, buf_id)
            return offset
        return None
    else:
        start_addr = self.find_free_space(size)
        if start_addr is not None:
            self._allocate_at(start_addr, size, buf_id)
            return start_addr
        return None

def free(self, buf_id: int) -> bool:
    """释放缓冲区并合并相邻空闲块（核心碎片处理）"""
    if buf_id not in self.buf_to_region:
        return False

    region = self.buf_to_region[buf_id]
    self.allocated_regions.remove(region)
    del self.buf_to_region[buf_id]
    self._insert_free_block(region.start, region.end)  # 合并碎片
    return True

def find_free_space(self, size: int) -> Optional[int]:
    """最佳适配算法：寻找最小能容纳的空闲块"""
    best_start = None
    best_len = None
    for (start, end) in self.free_blocks:
        length = end - start
        if length >= size:
            if best_len is None or length < best_len or (
                length == best_len and start < best_start):
                best_start = start
                best_len = length
    return best_start

def _allocate_at(self, start: int, size: int, buf_id: int):
    """在指定位置分配并拆分空闲块"""

```

```

end = start + size
# 拆分包含该区间的空闲块
for i, (fs, fe) in enumerate(self.free_blocks):
    if fs <= start and end <= fe:
        new_blocks = []
        if fs < start:
            new_blocks.append((fs, start))
        if end < fe:
            new_blocks.append((end, fe))
        self.free_blocks.pop(i)
        for nb in reversed(new_blocks):
            self.free_blocks.insert(i, nb)
        break
self.allocated_regions.append(CacheRegion(start, end,
    buf_id))
self.buf_to_region[buf_id] = CacheRegion(start, end,
    buf_id)

def _insert_free_block(self, start: int, end: int):
    """插入空闲块并合并相邻块（解决碎片问题）"""
    if start >= end:
        return
    idx = 0
    while idx < len(self.free_blocks) and self.free_blocks[
        idx][0] < start:
        idx += 1
    self.free_blocks.insert(idx, (start, end))
    self._merge_around(idx) # 合并前后相邻块

def get_largest_free_block(self) -> int:
    """获取最大连续空闲块（判断是否需要SPILL的关键指标）"""
    return max((fe - fs for fs, fe in self.free_blocks),
        default=0)

class SmartAddressAllocator:
    """智能分配器核心逻辑，减少碎片化"""

    def allocate_with_strategy(self, buf_id: int, size: int,

```

```

strategy: str = "segregated_fit") -> Optional[int]:
    """多策略分配接口，核心是碎片化感知算法"""
    if strategy == "fragmentation_aware":
        return self._fragmentation_aware_allocate(buf_id,
            size)
    elif strategy == "best_fit":
        return self._best_fit_allocate(buf_id, size)
    else:
        return self.segregated_allocator.allocate_segregated
            (buf_id, size)

def _fragmentation_aware_allocate(self, buf_id: int, size:
int) -> Optional[int]:
    """碎片化感知分配：选择对未来碎片影响最小的位置"""
    regions = sorted(self.cache_manager.allocated_regions,
        key=lambda r: r.start)
    candidates = []

    # 收集所有可能的分配位置
    if not regions or regions[0].start >= size:
        candidates.append((0, regions[0].start if regions
            else self.cache_manager.capacity))
    for i in range(len(regions)-1):
        gap_start, gap_end = regions[i].end, regions[i+1].
            start
        if gap_end - gap_start >= size:
            candidates.append((gap_start, gap_end))
    if regions and self.cache_manager.capacity - regions
        [-1].end >= size:
        candidates.append((regions[-1].end, self.
            cache_manager.capacity))

    if not candidates:
        return None

    # 选择碎片化得分最低的位置
    best_start = min(candidates, key=lambda c: self.
        _calculate_fragmentation_score(c[0], size, c[1]))[0]
    return self.cache_manager.allocate(buf_id, size,

```



```
best_start)
```

Problem2_check.py 核心代码

```
# 核心验证逻辑：缓存分配与SPILL操作校验

# 硬件缓存容量约束（题目表1）
# 本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
#   ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
#   7日。
CACHE_LIMITS = {'L1': 4096, 'UB': 1024, 'L0A': 256, 'L0B': 256,
                 'LOC': 512}

class BufferAllocation:
    """缓冲区分配信息"""
    def __init__(self, buf_id, offset, size, cache_type):
        self.buf_id = buf_id
        self.offset = offset
        self.size = size
        self.cache_type = cache_type

class ValidationCore:
    """核心验证器"""
    def check_cache_overflow(self, allocations):
        """检查缓存容量超限（核心约束1）"""
        errors = []
        for alloc in allocations:
            if alloc.cache_type not in CACHE_LIMITS:
                continue
            if alloc.offset + alloc.size > CACHE_LIMITS[alloc.
                cache_type]:
                errors.append(f"{alloc.cache_type}溢出: Buf{
                    alloc.buf_id}")
        return errors

    def check_address_overlap(self, allocations):
        """检查地址重叠（核心约束2）"""
        errors = []
        sorted_allocs = sorted(allocations, key=lambda x: x.
            offset)
```

```

    for i in range(len(sorted_allocs)-1):
        curr = sorted_allocs[i]
        next_alloc = sorted_allocs[i+1]
        if curr.offset + curr.size > next_alloc.offset:
            errors.append(f"重叠: Buf{curr.buf_id}与Buf{
                next_alloc.buf_id}")
    return errors

def calculate_spill_overhead(self, spill_ops, buffer_sizes,
    has_copy_in):
    """计算SPILL额外搬运量（核心指标）"""
    total = 0
    for op in spill_ops:
        size = buffer_sizes.get(op.buf_id, 0)
        if has_copy_in.get(op.buf_id, False):
            total += size # 仅SPILL_IN
        else:
            total += 2 * size # SPILL_OUT + SPILL_IN
    return total

```

C.3 第三问程序

Problem3_solve.py 核心代码

```

#本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
    ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
    7日。
class CacheRegion:
    """缓存区域基础结构"""
    def __init__(self, start: int, end: int, buf_id: int):
        self.start, self.end, self.buf_id = start, end, buf_id
    def size(self) -> int: return self.end - self.start

class CacheManager:
    """缓存核心管理：分配、释放、碎片处理"""
    def __init__(self, cache_type: str, capacity: int):
        self.cache_type = cache_type
        self.capacity = capacity
        self.allocated = [] # 已分配区域

```

```

self.buf_map = {}      # buf_id -> CacheRegion
self.free = [(0, capacity)]  # 空闲块 [(start, end)]

def find_free(self, size: int) -> Optional[int]:
    """最佳适配：找最小可用块"""
    best = None
    for s, e in self.free:
        if e - s >= size and (best is None or e - s < (self.
            free[best][1] - self.free[best][0])):
            best = self.free.index((s, e))
    return self.free[best][0] if best is not None else None

def allocate(self, buf_id: int, size: int) -> Optional[int]:
    """分配空间"""
    if buf_id in self.buf_map:
        return self.buf_map[buf_id].start
    start = self.find_free(size)
    if start:
        end = start + size
        self.allocated.append(CacheRegion(start, end, buf_id
            ))
        self.buf_map[buf_id] = CacheRegion(start, end,
            buf_id)
        self.free.remove((start, self.free[[i for i, (s, e)
            in enumerate(self.free) if s == start][0]][1]))
        if end < self.capacity:
            self.free.append((end, self.capacity))
            self.free.sort()
        return start
    return None

def free(self, buf_id: int) -> bool:
    """释放并合并空闲块"""
    if buf_id not in self.buf_map:
        return False
    region = self.buf_map[buf_id]
    self.allocated.remove(region)
    del self.buf_map[buf_id]
    self.free.append((region.start, region.end))

```

```

        self.free.sort()
        self._merge_free() # 合并相邻块
        return True

def _merge_free(self):
    """合并相邻空闲块"""
    i = 0
    while i < len(self.free)-1:
        if self.free[i][1] == self.free[i+1][0]:
            self.free[i] = (self.free[i][0], self.free[i+1][1])
            self.free.pop(i+1)
        else:
            i += 1

class SpillOp:
    """SPILL操作核心记录"""
    def __init__(self, buf_id: int, old_off: int, cache_type: str):
        self.buf_id = buf_id
        self.old_off = old_off # 原缓存地址
        self.cache_type = cache_type

class HardwareSim:
    """硬件仿真核心：缓存分配与SPILL处理"""
    CACHE_CAPS = {"L1": 4096, "UB": 1024} # 核心缓存容量

    def __init__(self):
        self.caches = {t: CacheManager(t, c) for t, c in self.CACHE_CAPS.items()}
        self.spills = [] # 记录SPILL操作

    def alloc(self, buf_id: int, size: int, cache_type: str) -> bool:
        """分配缓存，失败则触发SPILL"""
        cache = self.caches[cache_type]
        if cache.allocate(buf_id, size):

```

```

        return True
    # 分配失败，执行SPILL
    return self._spill_and_alloc(buf_id, size, cache_type)

def _spill_and_alloc(self, buf_id: int, size: int,
    cache_type: str) -> bool:
    """SPILL释放空间后重试分配"""
    cache = self.caches[cache_type]
    # 选择最早分配的缓冲区SPILL
    if not cache.allocated:
        return False
    spill_buf = sorted(cache.allocated, key=lambda r: r.
        start)[0].buf_id

    # 记录SPILL操作
    self.spills.append(SpillOp(spill_buf, cache.buf_map[
        spill_buf].start, cache_type))

    # 释放并重新分配
    cache.free(spill_buf)
    return cache.allocate(buf_id, size) is not None

```

Problem3_check.py 核心代码

```

# 核心缓存验证与SPILL操作关键逻辑

# 硬件缓存容量约束（核心配置）
# 本程序及代码是在人工智能工具辅助下完成的，人工智能工具名称：
# ChatGPT，版本:5，开发机构/公司:OpenAI，版本颁布日期2025年8月
# 7日。
CACHE_LIMITS = {'L1': 4096, 'UB': 1024, 'LOA': 256, 'LOB': 256,
    'LOC': 512}

def check_cache_overflow(allocations):
    """检查缓存容量超限（核心约束）"""
    errors = []
    for buf_id, (offset, size, cache_type) in allocations.items
        ():
            if offset + size > CACHE_LIMITS.get(cache_type, 0):
                errors.append(f"{cache_type} 溢出: Buf{buf_id}")

```

```

return errors

def check_address_overlap(allocations):
    """检查地址重叠（核心约束）"""
    errors = []
    sorted_allocs = sorted(allocations.items(), key=lambda x: x
        [1][0]) # 按地址排序
    for i in range(len(sorted_allocs)-1):
        buf1, (off1, size1, _) = sorted_allocs[i]
        buf2, (off2, _, _) = sorted_allocs[i+1]
        if off1 + size1 > off2:
            errors.append(f"重叠: Buf{buf1}与Buf{buf2}")
    return errors

def calculate_spill_overhead(spill_ops, buffer_sizes,
    has_copy_in):
    """计算SPILL额外搬运量（核心指标）"""
    total = 0
    for buf_id in spill_ops:
        size = buffer_sizes.get(buf_id, 0)
        # 有COPY_IN仅算1次搬入，否则算2次（搬出+搬入）
        total += size if has_copy_in.get(buf_id, False) else 2 *
            size
    return total

# 使用示例（核心流程）
if __name__ == "__main__":
    # 模拟数据: {buf_id: (offset, size, cache_type)}
    allocations = {1: (100, 512, 'L1'), 2: (600, 400, 'L1')}
    spill_ops = [1] # SPILL的缓冲区ID
    buffer_sizes = {1: 512, 2: 400}
    has_copy_in = {1: False, 2: True}

    # 执行核心验证
    overflow_errs = check_cache_overflow(allocations)
    overlap_errs = check_address_overlap(allocations)
    spill_overhead = calculate_spill_overhead(spill_ops,
        buffer_sizes, has_copy_in)

```

```
print("验证结果:", "通过" if not overflow_errs and not
      overlap_errs else "失败")
print("SPILL额外搬运量:", spill_overhead)
```